



Zeitenwende: Showing in Numbers How the 2022 Russian Invasion of Ukraine Began an Arms Reshuffling Across Europe

Clayton Greenwell

Dr. Nasim Anousheh

DSCI-D699 Graduate Independent Study in Data Science

July 2026

```
In [1]: # Google Colab instructions
#
# Runtime > Change Runtime Type > Runtime Version > 2025.07
# This cell should say you're on Python Version 3.11.

import sys
print(sys.version)
```

3.11.13 (main, Jun 4 2025, 08:57:29) [GCC 11.4.0]

1. About the Data

An extensive amount of data cleaning was done to transform the following datasets into one clean, usable spreadsheet:

- SIPRI Arms Transfer Database
- SIPRI Arms Industry Database
- SIPRI Military Expenditure Database
- Geopolitical Risk Index (Caldara & Iacoviello, 2022)
- Google Maps data (*aerial_distance_moscow_km*)

The Stockholm International Peace Research Institute (SIPRI) is a globally renowned think tank known for tracking the global arms industry. Their datasets have been used in countless peer-reviewed publications.

The SIPRI datasets obtained contained missing values. Linear interpolation was the constant method used to fill these missing values when appropriate. Several

features were created from calculations involving other features: *import_export_ratio*, *import_growth_rate*, *export_coverage_ratio*, *ME_spending_%_GDP_growth_rate*, and *export_growth_rate*. Aside from a later experiment with Gephi, all data used is broken down across the 4 years since the Russian invasion of Ukraine (2022-2025). All data is exclusive to European countries.

The cleaned dataset can be found here: https://github.com/ClaytonGreenwell/defense_spending_project/blob/main/final_submission/sipri_data1.xlsx

There are many columns in this dataset that function as a target, with an abundance of overlap:

- *import_TIV_millions*
- *import_export_ratio*
- *import_growth_rate*
- *export_coverage_ratio* (calculation relies on *import_TIV_millions*)
- *ME_spending_%_GDP*
- *ME_spending_%_GDP_lead_1_year*
- *ME_per_capita_in_USD*
- *ME_%_gov_budget*
- *ME_spending_%_GDP_growth_rate*

As an example, it's understandable that a country's arms imports (*import_TIV_millions*) will correspond directly to that country's military expenditure as a percentage of GDP (*ME_spending_%_GDP*). This target leakage is true of all above-mentioned columns and is dealt with appropriately in this report.

Some of the most interesting and usable features include:

- *export_TIV_millions*
- *export_growth_rate*
- *aerial_distance_moscow_km*
- *GPR_index*
- *is_in_NATO?*

- *top_supplier_country*
- *is_US_top_supplier_country?*
- *companies_in_sipri_top_100*

Many countries in *companies_in_sipri_top_100* contain floats. This is due to joint-EU defense companies that spread operations across multiple EU countries without assigning ownership to a single state.

Some notes on the choices made for visualizations throughout this report:

- The definition of “Europe” includes all countries that are members of the Council of Europe.
 - o Russia is included, though it was expelled from the Council in 2022.
 - o Belarus and Kosovo were included, despite not being members of the Council.
- Five “microstates” were excluded from all visualizations: Andorra, Liechtenstein, Monaco, San Marino, and Vatican City.
- Malta was included. Malta was not classified as a “microstate” despite its size and population for 3 reasons:
 - o It is a European Union member.
 - o Unlike the other 5 microstates, Malta doesn’t have a dramatic diplomatic dependence on other states.
 - o Malta’s location in the Mediterranean Sea is strategically important from a military defense perspective.
- Various disputed states were excluded: Northern Cyprus, Transnistria, Abkhazia, and South Ossetia.
- Article 41(2) of the Treaty on European Union prohibits the regular EU budget from being used for military defense purposes. Some EU states have contributed to a workaround agency called The European Peace Facility. The EPF purchases arms as a unified body. Many of these arms are given to Ukraine or are used to bolster a collective NATO/EU defense network (Cepparulo & Pasimeni, 2024). As this is a

collective agency that operates on the fringe of the EU, it's important to understand that imports and exports from this agency would not appear in import/export visualizations unless an actual country, (like Ukraine,) is receiving those arms as an import.

```
In [3]: # We'll need pycaret for creating our machine learning models later on. We'll  
# need to install pycaret before we import our data, as we'll otherwise lose  
# our data when restarting the session.
```

```
!pip install pycaret openpyxl
```

Collecting pycaret

Downloading pycaret-3.3.2-py3-none-any.whl.metadata (17 kB)

Requirement already satisfied: openpyxl in /usr/local/lib/python3.11/dist-packages (3.1.5)

Requirement already satisfied: ipython>=5.5.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (7.34.0)

Requirement already satisfied: ipywidgets>=7.6.5 in /usr/local/lib/python3.11/dist-packages (from pycaret) (7.7.1)

Requirement already satisfied: tqdm>=4.62.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (4.67.1)

Collecting numpy<1.27,>=1.21 (from pycaret)

Downloading numpy-1.26.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB)

61.0/61.0 kB 968.7 kB/s eta 0:00:00

Collecting pandas<2.2.0 (from pycaret)

Downloading pandas-2.1.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (18 kB)

Requirement already satisfied: jinja2>=3 in /usr/local/lib/python3.11/dist-packages (from pycaret) (3.1.6)

Collecting scipy<=1.11.4,>=1.6.1 (from pycaret)

Downloading scipy-1.11.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (60 kB)

60.4/60.4 kB 3.7 MB/s eta 0:00:00

Collecting joblib<1.4,>=1.2.0 (from pycaret)

Downloading joblib-1.3.2-py3-none-any.whl.metadata (5.4 kB)

Requirement already satisfied: scikit-learn>1.4.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (1.6.1)

Collecting pyod>=1.1.3 (from pycaret)

Downloading pyod-3.6.2-py3-none-any.whl.metadata (58 kB)

58.6/58.6 kB 4.2 MB/s eta 0:00:00

Requirement already satisfied: imbalanced-learn>=0.12.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (0.13.0)

Collecting category-encoders>=2.4.0 (from pycaret)

Downloading category_encoders-2.9.0-py3-none-any.whl.metadata (7.9 kB)

Requirement already satisfied: lightgbm>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (4.5.0)

Requirement already satisfied: numba>=0.55.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (0.60.0)

Requirement already satisfied: requests>=2.27.1 in /usr/local/lib/python3.11/dist-packages (from pycaret) (2.32.3)

Requirement already satisfied: psutil>=5.9.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (5.9.5)

Requirement already satisfied: markupsafe>=2.0.1 in /usr/local/lib/python3.11/dist-packages (from pycaret) (3.0.2)

Requirement already satisfied: importlib-metadata>=4.12.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (8.7.0)

Requirement already satisfied: nbformat>=4.2.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (5.10.4)

Requirement already satisfied: cloudpickle in /usr/local/lib/python3.11/dist-packages (from pycaret) (3.1.1)

Collecting deprecation>=2.1.0 (from pycaret)

Downloading deprecation-2.1.0-py2.py3-none-any.whl.metadata (4.6 kB)

Requirement already satisfied: xxhash in /usr/local/lib/python3.11/dist-packages

```

s (from pycaret) (3.5.0)
Collecting matplotlib<3.8.0 (from pycaret)
  Downloading matplotlib-3.7.5-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (5.7 kB)
Collecting scikit-plot>=0.3.7 (from pycaret)
  Downloading scikit_plot-0.3.7-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: yellowbrick>=1.4 in /usr/local/lib/python3.11/dist-packages (from pycaret) (1.5)
Requirement already satisfied: plotly>=5.14.0 in /usr/local/lib/python3.11/dist-packages (from pycaret) (5.24.1)
Collecting kaleido>=0.2.1 (from pycaret)
  Downloading kaleido-1.3.0-py3-none-any.whl.metadata (5.6 kB)
Collecting schemdraw==0.15 (from pycaret)
  Downloading schemdraw-0.15-py3-none-any.whl.metadata (2.2 kB)
Collecting plotly-resampler>=0.8.3.1 (from pycaret)
  Downloading plotly_resampler-0.11.0-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: statsmodels>=0.12.1 in /usr/local/lib/python3.11/dist-packages (from pycaret) (0.14.5)
Collecting sktime==0.26.0 (from pycaret)
  Downloading sktime-0.26.0-py3-none-any.whl.metadata (29 kB)
Collecting tbats>=1.1.3 (from pycaret)
  Downloading tbats-1.1.3-py3-none-any.whl.metadata (3.8 kB)
Collecting pmdarima>=2.0.4 (from pycaret)
  Downloading pmdarima-2.1.1-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (8.5 kB)
Requirement already satisfied: wurlitizer in /usr/local/lib/python3.11/dist-packages (from pycaret) (3.1.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from sktime==0.26.0->pycaret) (24.2)
Collecting scikit-base<0.8.0 (from sktime==0.26.0->pycaret)
  Downloading scikit_base-0.7.8-py3-none-any.whl.metadata (8.8 kB)
Collecting scikit-learn>1.4.0 (from pycaret)
  Downloading scikit_learn-1.4.2-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (11 kB)
Requirement already satisfied: et-xmlfile in /usr/local/lib/python3.11/dist-packages (from openpyxl) (2.0.0)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.11/dist-packages (from category-encoders>=2.4.0->pycaret) (1.0.1)
INFO: pip is looking at multiple versions of category-encoders to determine which version is compatible with other requirements. This could take a while.
Collecting category-encoders>=2.4.0 (from pycaret)
  Downloading category_encoders-2.8.1-py3-none-any.whl.metadata (7.9 kB)
  Downloading category_encoders-2.8.0-py3-none-any.whl.metadata (7.9 kB)
  Downloading category_encoders-2.7.0-py3-none-any.whl.metadata (7.9 kB)
Requirement already satisfied: sklearn-compat<1,>=0.1 in /usr/local/lib/python3.11/dist-packages (from imbalanced-learn>=0.12.0->pycaret) (0.1.3)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from imbalanced-learn>=0.12.0->pycaret) (3.6.0)
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.11/dist-packages (from importlib-metadata>=4.12.0->pycaret) (3.23.0)
Requirement already satisfied: setuptools>=18.5 in /usr/local/lib/python3.11/dist-packages (from ipython>=5.5.0->pycaret) (75.2.0)
Collecting jedi>=0.16 (from ipython>=5.5.0->pycaret)
  Downloading jedi-0.20.0-py2.py3-none-any.whl.metadata (23 kB)

```

Requirement already satisfied: decorator in /usr/local/lib/python3.11/dist-packages (from ipython>=5.5.0->pycaret) (4.4.2)

Requirement already satisfied: pickleshare in /usr/local/lib/python3.11/dist-packages (from ipython>=5.5.0->pycaret) (0.7.5)

Requirement already satisfied: traitlets>=4.2 in /usr/local/lib/python3.11/dist-packages (from ipython>=5.5.0->pycaret) (5.7.1)

Requirement already satisfied: prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from ipython>=5.5.0->pycaret) (3.0.51)

Requirement already satisfied: pygments in /usr/local/lib/python3.11/dist-packages (from ipython>=5.5.0->pycaret) (2.19.2)

Requirement already satisfied: backcall in /usr/local/lib/python3.11/dist-packages (from ipython>=5.5.0->pycaret) (0.2.0)

Requirement already satisfied: matplotlib-inline in /usr/local/lib/python3.11/dist-packages (from ipython>=5.5.0->pycaret) (0.1.7)

Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.11/dist-packages (from ipython>=5.5.0->pycaret) (4.9.0)

Requirement already satisfied: ipykernel>=4.5.1 in /usr/local/lib/python3.11/dist-packages (from ipywidgets>=7.6.5->pycaret) (6.17.1)

Requirement already satisfied: ipython-genutils~=0.2.0 in /usr/local/lib/python3.11/dist-packages (from ipywidgets>=7.6.5->pycaret) (0.2.0)

Requirement already satisfied: widgetsnbextension~=3.6.0 in /usr/local/lib/python3.11/dist-packages (from ipywidgets>=7.6.5->pycaret) (3.6.10)

Requirement already satisfied: jupyterlab-widgets>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from ipywidgets>=7.6.5->pycaret) (3.0.15)

Collecting choreographer>=1.3.0 (from kaleido>=0.2.1->pycaret)

Downloading choreographer-1.3.0-py3-none-any.whl.metadata (6.9 kB)

Collecting logistro>=1.0.8 (from kaleido>=0.2.1->pycaret)

Downloading logistro-2.0.1-py3-none-any.whl.metadata (3.9 kB)

Requirement already satisfied: orjson>=3.10.15 in /usr/local/lib/python3.11/dist-packages (from kaleido>=0.2.1->pycaret) (3.10.18)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib<3.8.0->pycaret) (1.3.2)

Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib<3.8.0->pycaret) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib<3.8.0->pycaret) (4.58.5)

Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib<3.8.0->pycaret) (1.4.8)

Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib<3.8.0->pycaret) (11.2.1)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib<3.8.0->pycaret) (3.2.3)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib<3.8.0->pycaret) (2.9.0.post0)

Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.11/dist-packages (from nbformat>=4.2.0->pycaret) (2.21.1)

Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.11/dist-packages (from nbformat>=4.2.0->pycaret) (4.24.0)

Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in /usr/local/lib/python3.11/dist-packages (from nbformat>=4.2.0->pycaret) (5.8.1)

Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.11/dist-packages (from numba>=0.55.0->pycaret) (0.43.0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas<2.2.0->pycaret) (2025.2)

Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/dist-packages (from pandas<2.2.0->pycaret) (2025.2)

Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly>=5.14.0->pycaret) (8.5.0)

Collecting dash>=2.11.0 (from plotly-resampler>=0.8.3.1->pycaret)

Downloading dash-4.4.0-py3-none-any.whl.metadata (11 kB)

Collecting tsdownsample>=0.1.3 (from plotly-resampler>=0.8.3.1->pycaret)

Downloading tsdownsample-0.1.5.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (8.1 kB)

Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in /usr/local/lib/python3.11/dist-packages (from pmdarima>=2.0.4->pycaret) (3.0.12)

INFO: pip is looking at multiple versions of pmdarima to determine which version is compatible with other requirements. This could take a while.

Collecting pmdarima>=2.0.4 (from pycaret)

Downloading pmdarima-2.1.0-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (8.5 kB)

Downloading pmdarima-2.0.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl.metadata (7.8 kB)

Requirement already satisfied: urllib3 in /usr/local/lib/python3.11/dist-packages (from pmdarima>=2.0.4->pycaret) (2.4.0)

INFO: pip is looking at multiple versions of pyod to determine which version is compatible with other requirements. This could take a while.

Collecting pyod>=1.1.3 (from pycaret)

Downloading pyod-3.6.1-py3-none-any.whl.metadata (59 kB)

59.6/59.6 kB 4.1 MB/s eta 0:00:00

Downloading pyod-3.6.0-py3-none-any.whl.metadata (58 kB)

58.2/58.2 kB 4.2 MB/s eta 0:00:00

Downloading pyod-3.5.4-py3-none-any.whl.metadata (57 kB)

58.0/58.0 kB 4.2 MB/s eta 0:00:00

Downloading pyod-3.5.2-py3-none-any.whl.metadata (58 kB)

58.0/58.0 kB 4.2 MB/s eta 0:00:00

Downloading pyod-3.5.1-py3-none-any.whl.metadata (58 kB)

58.0/58.0 kB 4.2 MB/s eta 0:00:00

Downloading pyod-3.5.0-py3-none-any.whl.metadata (57 kB)

57.3/57.3 kB 3.8 MB/s eta 0:00:00

Downloading pyod-3.4.0-py3-none-any.whl.metadata (57 kB)

57.3/57.3 kB 3.7 MB/s eta 0:00:00

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests>=2.27.1->pycaret) (3.4.2)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests>=2.27.1->pycaret) (3.10)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests>=2.27.1->pycaret) (2025.7.14)

Requirement already satisfied: platformdirs>=4.3.6 in /usr/local/lib/python3.11/dist-packages (from choreographer>=1.3.0->kaleido>=0.2.1->pycaret) (4.3.8)

Requirement already satisfied: simplejson>=3.19.3 in /usr/local/lib/python3.11/dist-packages (from choreographer>=1.3.0->kaleido>=0.2.1->pycaret) (3.20.1)

Requirement already satisfied: Flask<3.2,>=1.0.4 in /usr/local/lib/python3.11/dist-packages (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret) (3.1.1)

Requirement already satisfied: Werkzeug<3.2 in /usr/local/lib/python3.11/dist-packages (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret) (3.1.3)

Requirement already satisfied: typing_extensions>=4.1.1 in /usr/local/lib/python3.11/dist-packages (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret) (4.14.1)

Collecting retrying (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret)
 Downloading retrying-1.4.2-py3-none-any.whl.metadata (5.5 kB)
 Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.11/dist-packages (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret) (1.6.0)
 Collecting janus>=1.0.0 (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret)
 Downloading janus-2.0.0-py3-none-any.whl.metadata (5.3 kB)
 Requirement already satisfied: pydantic>=2.10 in /usr/local/lib/python3.11/dist-packages (from dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret) (2.11.7)
 Requirement already satisfied: debugpy>=1.0 in /usr/local/lib/python3.11/dist-packages (from ipykernel>=4.5.1->ipywidgets>=7.6.5->pycaret) (1.8.0)
 Requirement already satisfied: jupyter-client>=6.1.12 in /usr/local/lib/python3.11/dist-packages (from ipykernel>=4.5.1->ipywidgets>=7.6.5->pycaret) (6.1.12)
 Requirement already satisfied: pyzmq>=17 in /usr/local/lib/python3.11/dist-packages (from ipykernel>=4.5.1->ipywidgets>=7.6.5->pycaret) (24.0.1)
 Requirement already satisfied: tornado>=6.1 in /usr/local/lib/python3.11/dist-packages (from ipykernel>=4.5.1->ipywidgets>=7.6.5->pycaret) (6.4.2)
 Collecting parso<0.9.0,>=0.8.6 (from jedi>=0.16->ipython>=5.5.0->pycaret)
 Downloading parso-0.8.7-py2.py3-none-any.whl.metadata (8.2 kB)
 Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6->nbformat>=4.2.0->pycaret) (25.3.0)
 Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6->nbformat>=4.2.0->pycaret) (2025.4.1)
 Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6->nbformat>=4.2.0->pycaret) (0.36.2)
 Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6->nbformat>=4.2.0->pycaret) (0.26.0)
 Requirement already satisfied: ptyprocess>=0.5 in /usr/local/lib/python3.11/dist-packages (from pexpect>4.3->ipython>=5.5.0->pycaret) (0.7.0)
 Requirement already satisfied: wcwidth in /usr/local/lib/python3.11/dist-packages (from prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0->ipython>=5.5.0->pycaret) (0.2.13)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7->matplotlib<3.8.0->pycaret) (1.17.0)
 Requirement already satisfied: notebook>=4.4.1 in /usr/local/lib/python3.11/dist-packages (from widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (6.5.7)
 Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret) (1.9.0)
 Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret) (8.2.1)
 Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from Flask<3.2,>=1.0.4->dash>=2.11.0->plotly-resampler>=0.8.3.1->pycaret) (2.2.0)
 Requirement already satisfied: argon2-cffi in /usr/local/lib/python3.11/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (25.1.0)
 Requirement already satisfied: nbconvert>=5 in /usr/local/lib/python3.11/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (7.16.6)
 Requirement already satisfied: Send2Trash>=1.8.0 in /usr/local/lib/python3.11/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (1.8.3)

Requirement already satisfied: terminado>=0.8.3 in /usr/local/lib/python3.11/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidget s>=7.6.5->pycaret) (0.18.1)

Requirement already satisfied: prometheus-client in /usr/local/lib/python3.11/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidget s>=7.6.5->pycaret) (0.22.1)

Requirement already satisfied: nbclassic>=0.4.7 in /usr/local/lib/python3.11/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidget s>=7.6.5->pycaret) (1.3.1)

Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.11/dist-packages (from pydantic>=2.10->dash>=2.11.0->plotly-resample r>=0.8.3.1->pycaret) (0.7.0)

Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.11/dist-packages (from pydantic>=2.10->dash>=2.11.0->plotly-resample r>=0.8.3.1->pycaret) (2.33.2)

Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from pydantic>=2.10->dash>=2.11.0->plotly-resample r>=0.8.3.1->pycaret) (0.4.1)

Requirement already satisfied: notebook-shim>=0.2.3 in /usr/local/lib/python3.11/dist-packages (from nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (0.2.4)

Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.11/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (4.13.4)

Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.11/dist-packages (from bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (6.2.0)

Requirement already satisfied: defusedxml in /usr/local/lib/python3.11/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (0.7.1)

Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.11/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (0.3.0)

Requirement already satisfied: mistune<4,>=2.0.3 in /usr/local/lib/python3.11/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (3.1.3)

Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.11/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (0.10.2)

Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.11/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (1.5.1)

Requirement already satisfied: argon2-cffi-bindings in /usr/local/lib/python3.11/dist-packages (from argon2-cffi->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (21.2.0)

Requirement already satisfied: webencodings in /usr/local/lib/python3.11/dist-packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (0.5.1)

Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (1.4.0)

Requirement already satisfied: jupyter-server<3,>=1.8 in /usr/local/lib/python3.11/dist-packages (from notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~=3.6.0->ipywidgets>=7.6.5->pycaret) (1.16.0)

Requirement already satisfied: cffi>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from argon2-cffi-bindings->argon2-cffi->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets>=7.6.5->pycaret) (1.17.1)

Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/dist-packages (from beautifulsoup4->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets>=7.6.5->pycaret) (2.7)

Requirement already satisfied: pyparser in /usr/local/lib/python3.11/dist-packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets>=7.6.5->pycaret) (2.22)

Requirement already satisfied: anyio>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets>=7.6.5->pycaret) (4.9.0)

Requirement already satisfied: websocket-client in /usr/local/lib/python3.11/dist-packages (from jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets>=7.6.5->pycaret) (1.8.0)

Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.11/dist-packages (from anyio>=3.1.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets>=7.6.5->pycaret) (1.3.1)

Downloading pycaret-3.3.2-py3-none-any.whl (486 kB)
486.1/486.1 kB 4.8 MB/s eta 0:00:00

Downloading schemdraw-0.15-py3-none-any.whl (106 kB)
106.8/106.8 kB 18.2 MB/s eta 0:00:00

0

Downloading sktime-0.26.0-py3-none-any.whl (21.8 MB)
21.8/21.8 MB 60.3 MB/s eta 0:00:00

Downloading category_encoders-2.7.0-py3-none-any.whl (85 kB)
85.4/85.4 kB 14.7 MB/s eta 0:00:00

Downloading deprecation-2.1.0-py2.py3-none-any.whl (11 kB)

Downloading joblib-1.3.2-py3-none-any.whl (302 kB)
302.2/302.2 kB 39.9 MB/s eta 0:00:00

0

Downloading kaleido-1.3.0-py3-none-any.whl (55 kB)
55.6/55.6 kB 9.4 MB/s eta 0:00:00

Downloading matplotlib-3.7.5-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (11.6 MB)
11.6/11.6 MB 67.5 MB/s eta 0:00:00

Downloading numpy-1.26.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.3 MB)
18.3/18.3 MB 59.4 MB/s eta 0:00:00

Downloading pandas-2.1.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.2 MB)
12.2/12.2 MB 65.5 MB/s eta 0:00:00

Downloading plotly_resampler-0.11.0-py3-none-any.whl (82 kB)
82.7/82.7 kB 14.3 MB/s eta 0:00:00

Downloading pmdarima-2.0.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl (2.2 MB)
2.2/2.2 MB 138.4 MB/s eta 0:00:00

Downloading pyod-3.4.0-py3-none-any.whl (383 kB)
383.6/383.6 kB 47.9 MB/s eta 0:00:00

0

Downloading scikit_learn-1.4.2-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.1 MB)

```

_____ 12.1/12.1 MB 37.5 MB/s eta 0:00:00
Downloading scikit_plot-0.3.7-py3-none-any.whl (33 kB)
Downloading scipy-1.11.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (36.4 MB)
_____ 36.4/36.4 MB 28.7 MB/s eta 0:00:00
Downloading tbats-1.1.3-py3-none-any.whl (44 kB)
_____ 44.0/44.0 kB 7.9 MB/s eta 0:00:00
Downloading choreographer-1.3.0-py3-none-any.whl (52 kB)
_____ 52.6/52.6 kB 8.1 MB/s eta 0:00:00
Downloading dash-4.4.0-py3-none-any.whl (8.9 MB)
_____ 8.9/8.9 MB 78.9 MB/s eta 0:00:00
Downloading jedi-0.20.0-py2.py3-none-any.whl (4.9 MB)
_____ 4.9/4.9 MB 84.2 MB/s eta 0:00:00
Downloading logistro-2.0.1-py3-none-any.whl (8.6 kB)
Downloading scikit_base-0.7.8-py3-none-any.whl (130 kB)
_____ 130.1/130.1 kB 19.6 MB/s eta 0:00:00
0
Downloading tsdownsample-0.1.5.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.3 MB)
_____ 1.3/1.3 MB 87.4 MB/s eta 0:00:00
Downloading janus-2.0.0-py3-none-any.whl (12 kB)
Downloading parso-0.8.7-py2.py3-none-any.whl (107 kB)
_____ 107.0/107.0 kB 19.0 MB/s eta 0:00:00
0
Downloading retrying-1.4.2-py3-none-any.whl (10 kB)
Installing collected packages: scikit-base, schemdraw, retrying, parso, numpy, logistro, joblib, janus, deprecation, tsdownsample, scipy, pandas, jedi, choreographer, scikit-learn, matplotlib, kaleido, dash, sktime, scikit-plot, pyod, pm darima, plotly-resampler, category-encoders, tbats, pycaret
Attempting uninstall: parso
  Found existing installation: parso 0.8.4
  Uninstalling parso-0.8.4:
    Successfully uninstalled parso-0.8.4
Attempting uninstall: numpy
  Found existing installation: numpy 2.0.2
  Uninstalling numpy-2.0.2:
    Successfully uninstalled numpy-2.0.2
Attempting uninstall: joblib
  Found existing installation: joblib 1.5.1
  Uninstalling joblib-1.5.1:
    Successfully uninstalled joblib-1.5.1
Attempting uninstall: scipy
  Found existing installation: scipy 1.15.3
  Uninstalling scipy-1.15.3:
    Successfully uninstalled scipy-1.15.3
Attempting uninstall: pandas
  Found existing installation: pandas 2.2.2
  Uninstalling pandas-2.2.2:
    Successfully uninstalled pandas-2.2.2
Attempting uninstall: scikit-learn
  Found existing installation: scikit-learn 1.6.1
  Uninstalling scikit-learn-1.6.1:
    Successfully uninstalled scikit-learn-1.6.1
Attempting uninstall: matplotlib

```

```

Found existing installation: matplotlib 3.10.0
Uninstalling matplotlib-3.10.0:
  Successfully uninstalled matplotlib-3.10.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 2.1.4 which is incompatible.
mizani 0.13.5 requires pandas>=2.2.0, but you have pandas 2.1.4 which is incompatible.
opencv-python-headless 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
plotnine 0.14.6 requires matplotlib>=3.8.0, but you have matplotlib 3.7.5 which is incompatible.
plotnine 0.14.6 requires pandas>=2.2.0, but you have pandas 2.1.4 which is incompatible.
thinc 8.3.6 requires numpy<3.0.0,>=2.0.0, but you have numpy 1.26.4 which is incompatible.
tsfresh 0.21.0 requires scipy>=1.14.0; python_version >= "3.10", but you have scipy 1.11.4 which is incompatible.
umap-learn 0.5.9.post2 requires scikit-learn>=1.6, but you have scikit-learn 1.4.2 which is incompatible.
Successfully installed category-encoders-2.7.0 choreographer-1.3.0 dash-4.4.0 deprecation-2.1.0 janus-2.0.0 jedi-0.20.0 joblib-1.3.2 kaleido-1.3.0 logistro-2.0.1 matplotlib-3.7.5 numpy-1.26.4 pandas-2.1.4 parso-0.8.7 plotly-resampler-0.11.0 pmdarima-2.0.4 pycaret-3.3.2 pyod-3.4.0 retrying-1.4.2 schemdraw-0.15 scikit-base-0.7.8 scikit-learn-1.4.2 scikit-plot-0.3.7 scipy-1.11.4 sktime-0.26.0 tbats-1.1.3 tsdownsample-0.1.5.1

```

```

In [1]: # Due to installing pycaret, before running this cell:
        # Runtime > Restart Session

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

```

```

In [2]: df = pd.read_excel("sipri_data1.xlsx")

```

2. Tableau Visualization I: Arms Exports

Europe's Engine

Figure 1:

<https://public.tableau.com/app/profile/clayton.greenwell/viz/EuropeAverageAnnualArmsExportTIVs20222025/Sheet1>

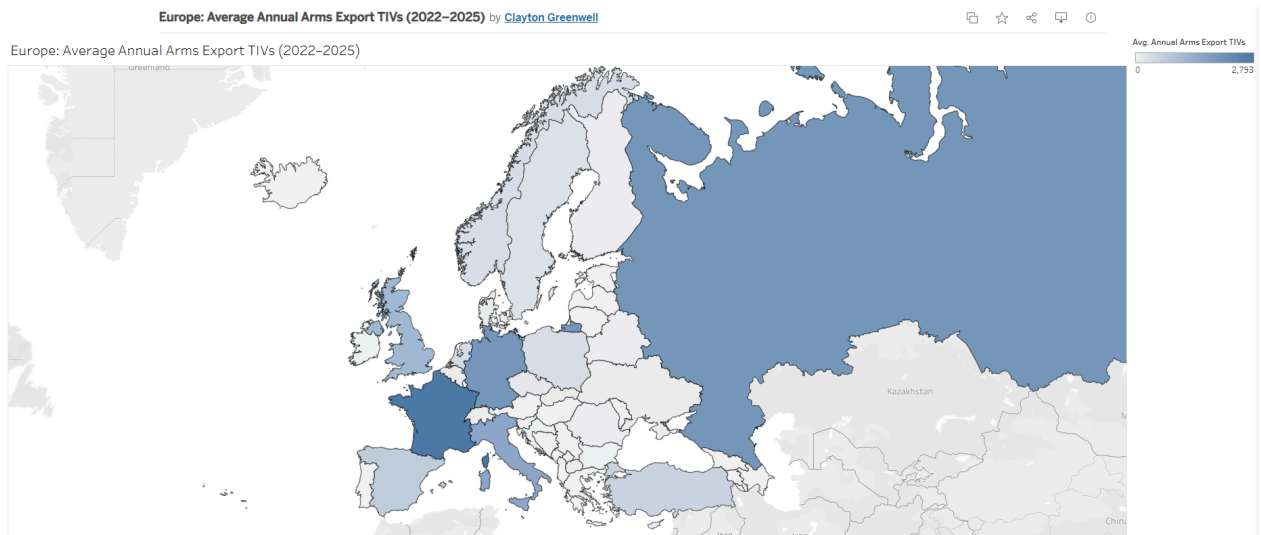
The above link is interactive. Here is a static image of Figure 1:

```

In [3]: from IPython.display import Image, display

```

```
display(Image(filename="export_tableau.png", width=1100))
```



Some notes on this visualization:

- This visualization averages annual arms import data from 2022 – 2025. (The wider Russian ground invasion of Ukraine began in 2022).
- The unit of measurement used in this visualization is Trend Indicator Values, commonly known as TIV units. TIV units are the gold standard for arms measurements because they help normalize what would otherwise be a very difficult calculation. (Keep in mind that using euros or dollars sold wouldn't be practical in a long-term study, as there were long periods during the Cold War that the US and USSR gave away weapons to other countries for free). As a measurement tool, TIV units standardize production costs by evaluating raw capability and resources required to build hardware (SIPRI, n.d.).

Germany, France, Italy, and the United Kingdom are strong European engine economies that have the capabilities to create and export an incredible number of arms. These 4 countries together have an annual output of 7.47 billion TIV units since the start of the war. SIPRI compiles a yearly list of the Top 100 Global Defense Companies. Germany, France, Italy, and the UK have collectively comprised roughly 20% of the companies listed since the outbreak of war in 2022 (SIPRI, n.d.).

Despite tough foreign sanctions and the ongoing domestic needs of immediate warfare, Russia has managed to export an average of 2 billion TIV units a year since the start of the war. Arms sales to countries like India have helped to bolster Russia's struggling economy (SIPRI, n.d.). On the world market, many of Russia's most sought-after weapons rely in some small part on small components

manufactured in the US, the UK, and other Western countries that are attempting to enforce sanctions and export control restrictions (George & Sandler, 2022). Export control restrictions are meant to throttle Russia's supply of these dual-use components. Russia's inability to access advanced computer chip technology is a critical chokepoint affecting Russia's arms industry (George & Sandler, 2022).

3. Tableau Visualization II: Arms Imports

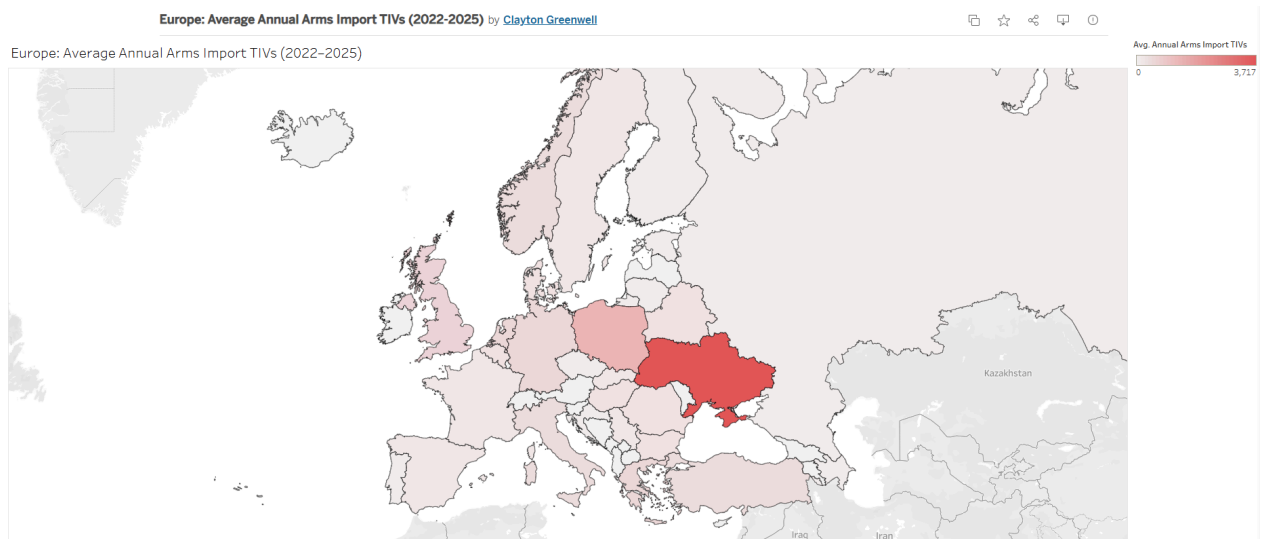
Russia Anxiety in Poland

Figure 2:

<https://public.tableau.com/app/profile/clayton.greenwell/viz/EuropeAverageAnnualArmsImportTIVs2022-2025/Sheet1>

The above link is interactive. Here is a static image of Figure 1:

```
In [4]: display(Image(filename="import_tableau.png", width=1100))
```



Ukraine's massive annual import of 3.72 billion TIV units worth of arms sits as an outlier, massively tipping the color scale and leaving other countries unsaturated.

Despite this outlier, important information can be gleaned by country shading that isn't as immediately stark. Poland's annual average import of 1.38 billion arms is reflective of the emerging role that Poland now plays as Europe's rising military power. On the surface, Poland is a country at peace containing many attributes that make war seem incredibly unlikely. A close partner of the United States, Poland is a member of NATO and therefore has considerable shielding due to NATO's Article 5 collective defense obligation.

The bigger picture is that NATO membership alone doesn't ease or erase recent memory. Unlike most Western European countries, Poland has a recent history of being subjected to Russian occupation and undue influence. Russia, (as its predecessor state, the USSR,) had some sort of troop presence inside Poland's borders from 1939 to 1993, stretching 54 years of selective leverage (Nawrocki, 2023).

There exists a strong belief in Poland that Russia will use their borders to test the seriousness of NATO's Article 5. As a country, Poland sits geographically between several pain points: a shared border with Ukraine, a shared border with Russia's exclave of Kaliningrad, a shared border with Russia's vassal state Belarus, and the Suwałki Gap corridor connecting Russia from Belarus to its exclave. Seeing these four fragile border vulnerabilities, Poland has prioritized its territorial integrity, beefing up its military stockpiles in preparation for Russian maneuvering (Tian et al., 2023).

One could surmise from this visualization and the previous export visualization that it would make sense for a country like Poland to take the domestic manufacture of arms much more seriously, being that they are both at peace and in a relatively high need for dramatically high imports. This gets to one of the known realities of arms exporting: arms exporting is a tight circle that new countries do not join. Developing economies do not develop arms-making engines alongside their growing industry. A small group of players dominates the arms exporting market with no room for new players. The vast majority of countries in the world will always be importers, while a select few will remain export powerhouses (Lebacher et al., 2021)

4. Gephi Network Visualizations: Europe's Splintering Arms Supply Network

How the 2022 Invasion of Ukraine Disrupted Europe's Arms Supply Network

Figure 3:

```
In [5]: display(  
    Image(  
        filename="top_arms_supplier_relationships_before_invasion_2018_2021.pr  
        width=1100  
    )  
)
```

Top Arms Supplier Relationships Before Invasion (2018-2021)

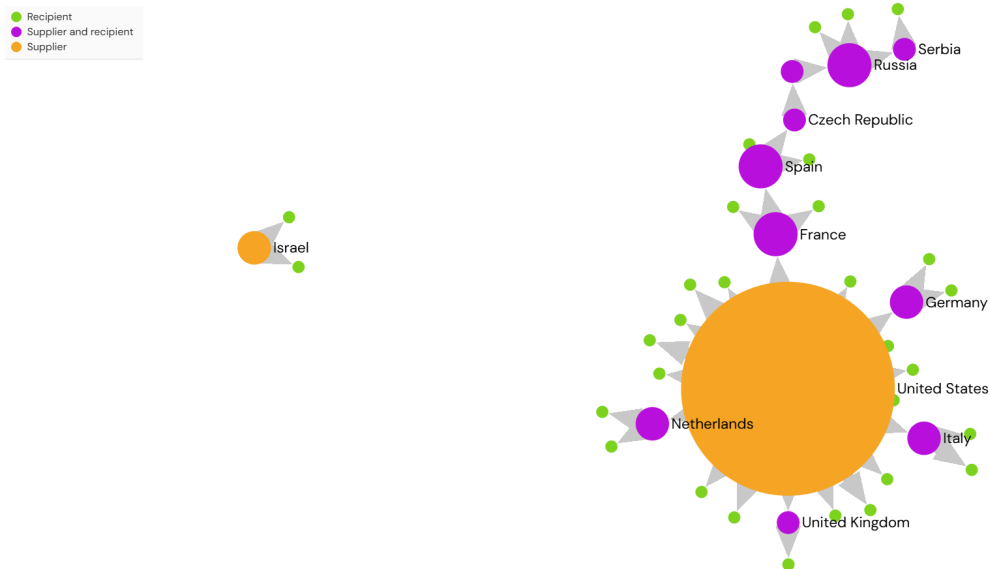
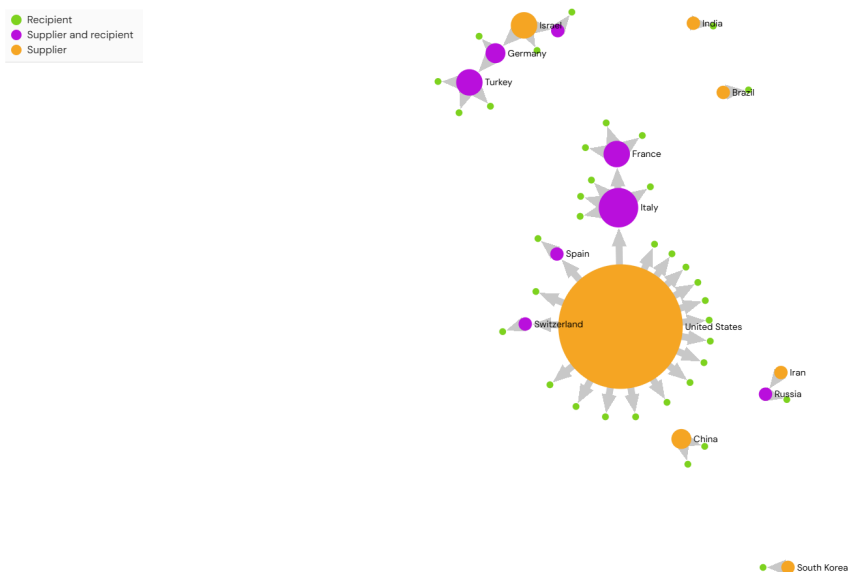


Figure 4:

```
In [6]: display(
    Image(
        filename="top_arms_supplier_relationships_after_invasion_2022_2025.png",
        width=1100
    )
)
```

Top Arms Supplier Relationships After Invasion (2022-2025)



Gephi is a powerful visualization tool for building network models. Perfect for visualizing supplier-recipient relationships, it gives us unique insight into how

Russia's 2022 invasion of Ukraine disrupted a surprisingly well-connected European arms network.

With the invasion beginning in 2022, we have 4 full years of post-invasion data (2022-2025). Using these 4 years, we look at the total TIV units imported by every European country to determine its top supplier (SIPRI, n.d.). This metric relies on *plurality*. For example, a country could have a very diverse supplier profile with their top supplier only making up 30% of their imports. But that top supplier would still be the most important defense relationship—and it would be reflected in this metric. Figure 3 shows the 4 years leading up to the war (2018-2021) to give us an idea of what the supplier-recipient relationships looked like before the disruption.

In Figure 3, we see a remarkable amount of connectedness amongst nearly all European countries and their suppliers. The United States stands out as a giant top supplier for the most countries: a whopping 19. Even *Russia* is connected in this arms supply network, although for unusual reasons. To our surprise, Russia's top arms supplier before the outbreak of war was in fact *Ukraine*. This is due to a concept called *dual-use technology*, where a technology sold for civilian purposes can be repurposed for military use. SIPRI's 2018 data shows TIV transfers of Antonov An-148-100E transport jet parts from Ukraine to Russia (SIPRI, n.d.) (UNIAN, 2019). Evidence is unclear as to whether Russia has used these jets for military purposes since the start of the invasion, but the example stands as a reminder that dual-use technology creates an incredible amount of overlap between the aerospace and defense industries.

Figure 4 shows this same European arms supply network become incredibly fragmented after the outbreak of the full-scale invasion. We see the existence of 7 unconnected networks: *the United States, Israel, India, Brazil, Iran, China, and South Korea*. The disparity in connectedness between the pre-war Figure 3 and the post-invasion Figure 4 shows us a European arms supply network that has splintered, with governments realigning their priorities and alliances.

The pre-war arms network's surprising connectedness is not coincidental. Students of history may remember the First World War as a baffling conflict with a laundry list of reasons for its beginning. But history teachers hold on to a single sentence that best sums up the powder keg that began the Great War: *The system of alliances that had kept Europe at peace had broken down*. We can view the remarkable connectedness of the 2018-2021 European arms network modeled in Figure 3 as an individual system of alliances exclusive to weaponry and defense. It cannot be inferred that the fragmentation that is reflected in the post-invasion Figure 4 will begin a global conflict like the First World War. But it is a grave illustration of the seriousness and significance of this current conflict: Europe's first

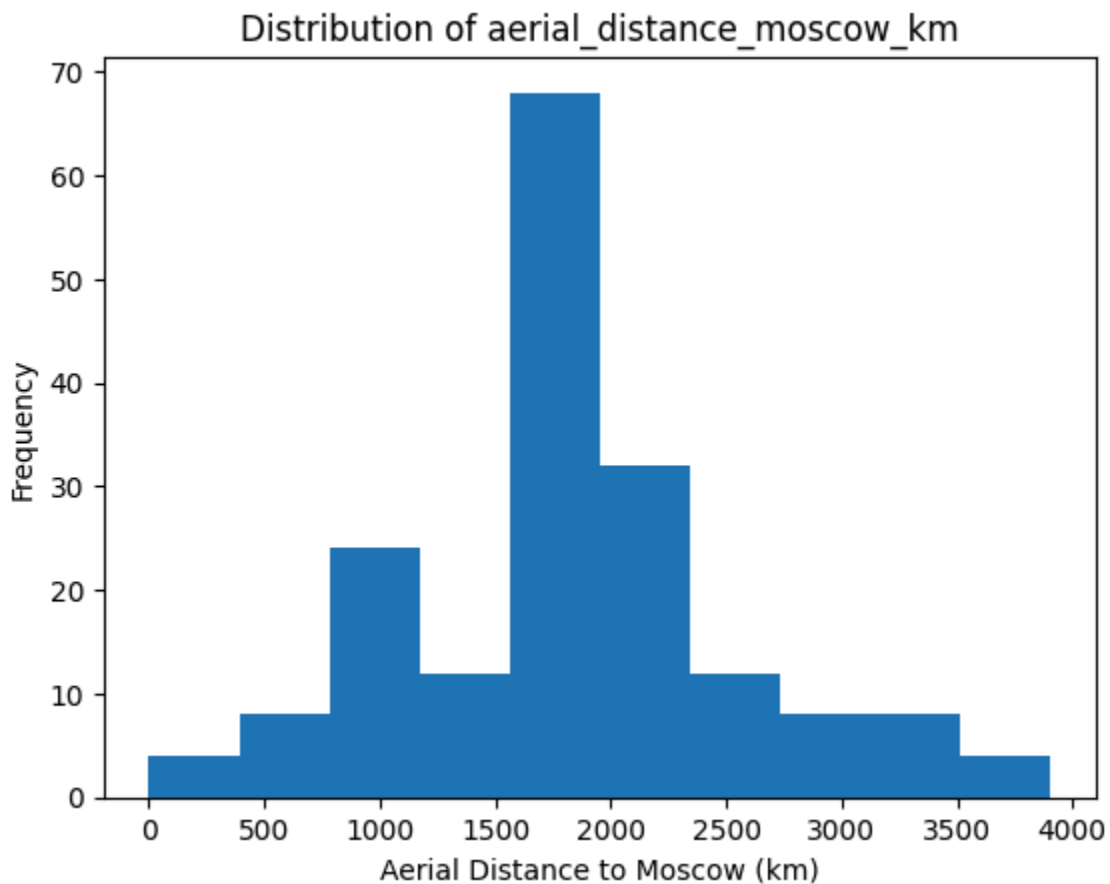
large, conventional land war since the World War period.

5. Histograms, Heatmap, and Pearson Correlation Coefficients

Digging in to the coding section, we start exploring our data with some simple statistics and visualizations.

```
In [7]: # Our only feature that follows a Normal distribution
```

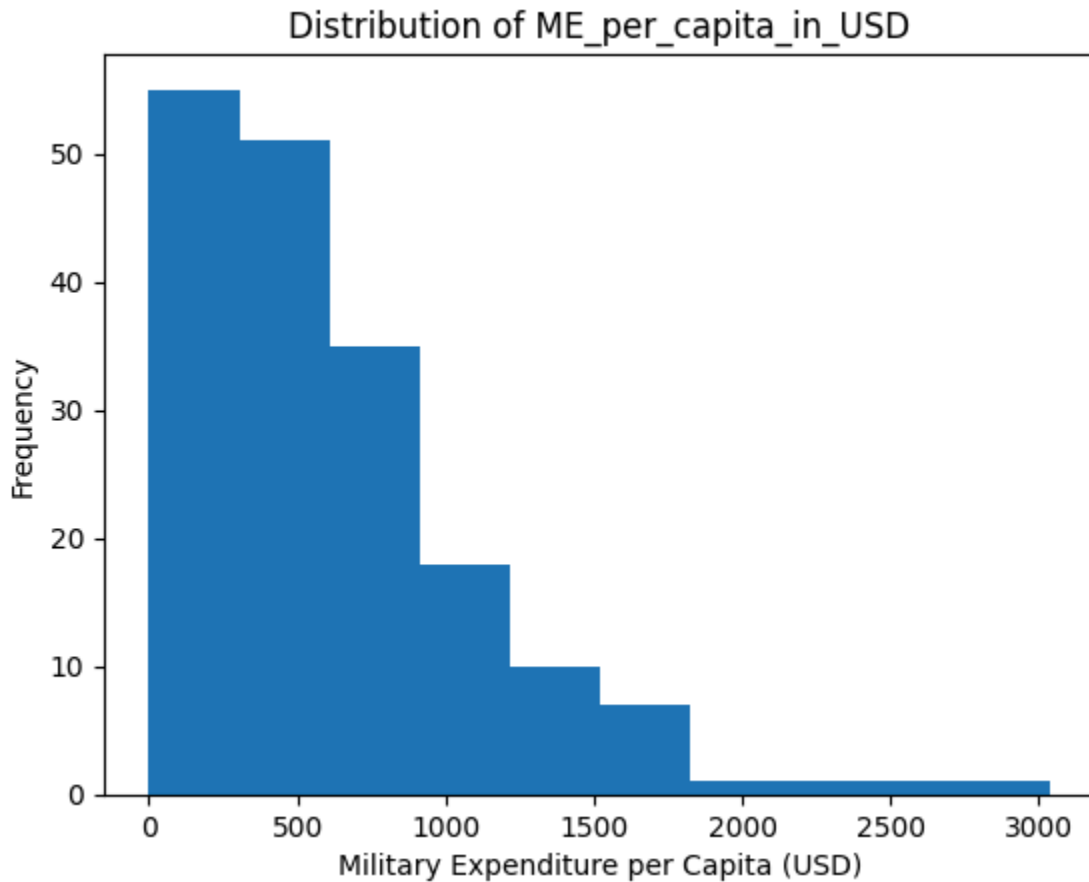
```
plt.hist(df["aerial_distance_moscow_km"])
plt.title("Distribution of aerial_distance_moscow_km")
plt.xlabel("Aerial Distance to Moscow (km)")
plt.ylabel("Frequency")
plt.show()
```



```
In [8]: # An example of a feature with a skewed distribution.
# Shows us that a lot of European countries spend very little per capita.
```

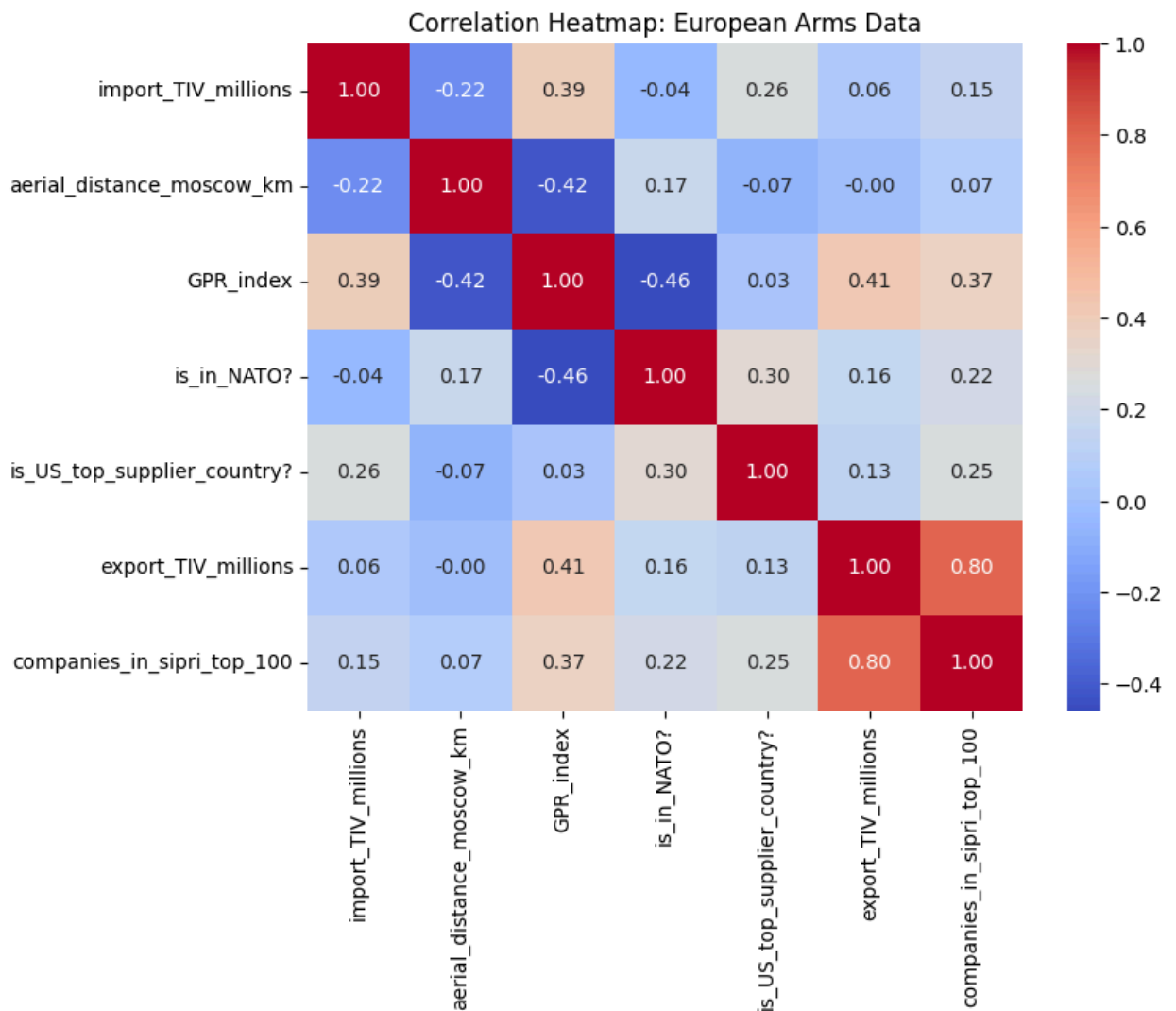
```
plt.hist(df["ME_per_capita_in_USD"])
plt.title("Distribution of ME_per_capita_in_USD")
plt.xlabel("Military Expenditure per Capita (USD)")
plt.ylabel("Frequency")
```

```
plt.show()
```



In [9]: *# Heatmap of our features*

```
features = [  
    'import_TIV_millions',  
    'aerial_distance_moscow_km',  
    'GPR_index',  
    'is_in_NATO?',  
    'is_US_top_supplier_country?',  
    'export_TIV_millions',  
    'companies_in_sipri_top_100'  
]  
  
heatmap_df = df[features].corr()  
  
plt.figure(figsize=(8, 6))  
sns.heatmap(heatmap_df, annot=True, cmap='coolwarm', fmt=".2f")  
plt.title('Correlation Heatmap: European Arms Data')  
plt.show()
```



```
In [10]: # Pearson Correlation Coefficients
```

```
# Filter down to only the year 2023, and only NATO members.
NATO_2023 = df[(df["year"] == 2023) & (df["is_in_NATO?"] == 1)]

distance_vs_ME = (
    NATO_2023["aerial_distance_moscow_km"]
    .corr(NATO_2023["ME_spending_%_GDP_lead_1_year"])
)

GPR_vs_top100 = (
    NATO_2023["GPR_index"]
    .corr(NATO_2023["companies_in_sipri_top_100"])
)

corr_table = pd.DataFrame({
    "Description": [
        "Distance From Moscow vs. ME % GDP (NATO, 2024)",
        # The reason that the label says (NATO, 2024) is that we're using
        # ME_spending_%_GDP_lead_1_year in the correlation.
        "GPR Index vs. Top 100 Defense Companies (NATO, 2023)",
    ]
})
```

```

    ],
    "Pearson Correlation": [
        distance_vs_ME,
        GPR_vs_top100
    ]
})

print(corr_table)

```

	Description	Pearson Correlation
0	Distance From Moscow vs. ME % GDP (NATO, 2024)	-0.672786
1	GPR Index vs. Top 100 Defense Companies (NATO,...	0.881429

```

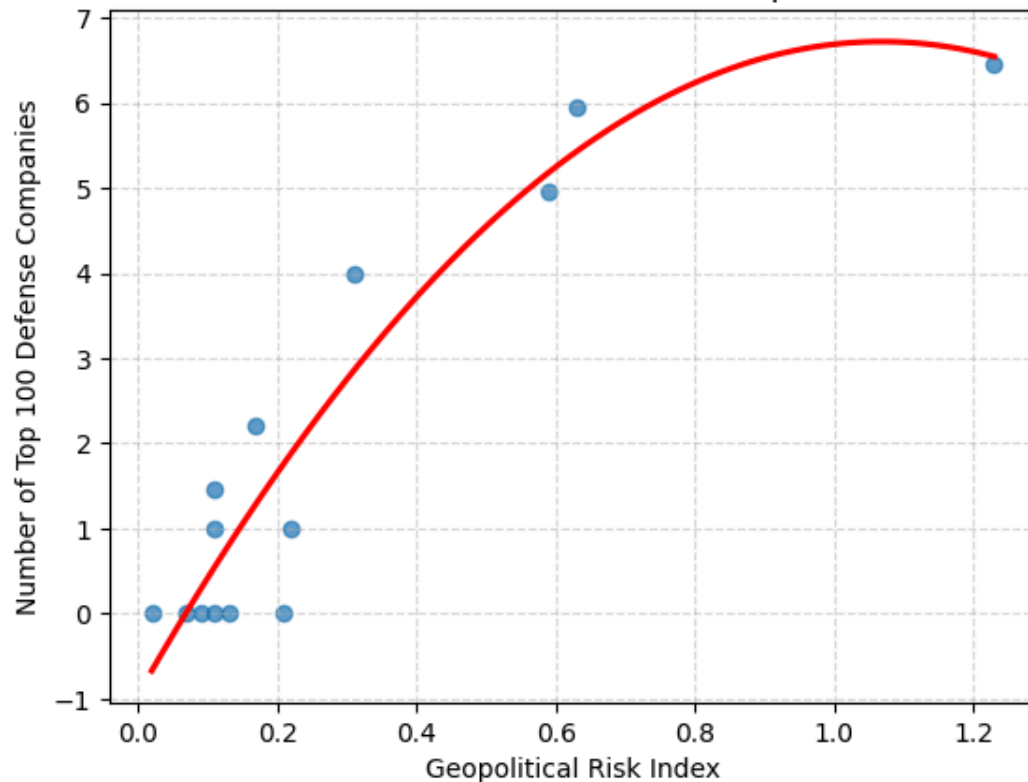
In [11]: # We're going to plot the better of the two correlations.
         # This will be polynomial regression.

sns.regplot(
    data=NATO_2023,
    x="GPR_index",
    y="companies_in_sipri_top_100",
    order=2,  # This will make it look like 2nd-degree polynomial regression
    ci=None,
    scatter_kws={"alpha": 0.7},
    line_kws={"color": "red"}
)

plt.xlabel("Geopolitical Risk Index")
plt.ylabel("Number of Top 100 Defense Companies")
plt.title("NATO Members (2023): GPR Index vs. Number of Top 100 Defense Compar")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()

```

NATO Members (2023): GPR Index vs. Number of Top 100 Defense Companies



We explore this here instead of in the regression section because we have enough data points to make a neat visualization, but not enough for the more complex regression modeling coming later. GPR Index as a feature has lots of missingness, as not all European countries were given values.

6. Regression Modeling: The Year That Rebalanced Europe

How 2023 Became a Powerful Predictor of Military Spending

```
In [12]: from pycaret.regression import *  
  
# We'll build the first model pipeline.  
  
regression_pipeline = setup(  
    data=df,  
    target='ME_spending_%_GDP',  
    categorical_features=['year'],  
    ignore_features=[  
        'GPR_index',  
        'companies_in_sipri_top_100',  
        'is_in_NATO?',  
        'export_coverage_ratio',  
        'export_TIV_millions',
```

```
        'is_US_top_supplier_country?',
        'top_supplier_country',
        'import_TIV_millions',
        'country',
        'ME_spending_%_GDP_lead_1_year',
        'ME_per_capita_in_USD',
        'ME_%_gov_budget',
        'ME_spending_%_GDP_growth_rate',
        'import_growth_rate',
        'import_export_ratio',
        'export_growth_rate',
    ],
    numeric_imputation='median',
    #categorical_imputation='constant',
    normalize=True,
    session_id=42,
    train_size=0.8,
    fold = 5
)
```

	Description	Value
0	Session id	42
1	Target	ME_spending_%_GDP
2	Target type	Regression
3	Original data shape	(180, 19)
4	Transformed data shape	(180, 6)
5	Transformed train set shape	(144, 6)
6	Transformed test set shape	(36, 6)
7	Ignore features	16
8	Numeric features	1
9	Categorical features	1
10	Rows with missing values	70.0%
11	Preprocess	True
12	Imputation type	simple
13	Numeric imputation	median
14	Categorical imputation	mode
15	Maximum one-hot encoding	25
16	Encoding method	None
17	Normalize	True
18	Normalize method	zscore
19	Fold Generator	KFold
20	Fold Number	5
21	CPU Jobs	-1
22	Use GPU	False
23	Log Experiment	False
24	Experiment Name	reg-default-name
25	USI	d48b

```
In [13]: best_model = compare_models()
```

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE	TT (Sec)
xgboost	Extreme Gradient Boosting	0.0048	0.0001	0.0084	0.8164	0.0074	0.2512	0.0700
huber	Huber Regressor	0.0116	0.0017	0.0320	0.0284	0.0278	0.5901	0.0440
lasso	Lasso Regression	0.0147	0.0018	0.0342	-0.3794	0.0300	1.0405	0.0420
dummy	Dummy Regressor	0.0147	0.0018	0.0342	-0.3794	0.0300	1.0405	0.0400
llar	Lasso Least Angle Regression	0.0147	0.0018	0.0342	-0.3794	0.0300	1.0405	0.0400
en	Elastic Net	0.0147	0.0018	0.0342	-0.3794	0.0300	1.0405	0.0660
omp	Orthogonal Matching Pursuit	0.0157	0.0017	0.0344	-0.7290	0.0293	0.8826	0.0420
br	Bayesian Ridge	0.0153	0.0018	0.0344	-0.7490	0.0297	0.9028	0.0400
et	Extra Trees Regressor	0.0154	0.0021	0.0364	-0.8341	0.0319	0.6916	0.1180
knn	K Neighbors Regressor	0.0152	0.0019	0.0370	-1.0720	0.0325	0.7914	0.0520
ridge	Ridge Regression	0.0169	0.0018	0.0360	-1.3943	0.0307	0.9553	0.0560
lr	Linear Regression	0.0169	0.0018	0.0360	-1.4217	0.0307	0.9582	1.3740
lar	Least Angle Regression	0.0169	0.0018	0.0360	-1.4217	0.0307	0.9582	0.0620
lightgbm	Light Gradient Boosting Machine	0.0158	0.0019	0.0370	-1.5086	0.0326	0.9289	0.1380
gbr	Gradient Boosting Regressor	0.0073	0.0007	0.0216	-1.5598	0.0182	0.3474	0.0720
rf	Random Forest Regressor	0.0100	0.0009	0.0271	-2.2915	0.0234	0.4610	0.1440
ada	AdaBoost Regressor	0.0120	0.0015	0.0304	-3.4243	0.0264	0.7545	0.0760
dt	Decision Tree Regressor	0.0098	0.0018	0.0312	-4.4893	0.0267	0.4216	0.0400

	Model	MAE	MSE	RMSE	R2	RMSLE	MAPE	TT (Sec)
par	Passive Aggressive Regressor	0.0513	0.0049	0.0689	-24.6622	0.0593	3.4693	0.0400

```
In [14]: xgboost_model = create_model('xgboost')
```

	MAE	MSE	RMSE	R2	RMSLE	MAPE
Fold						
0	0.0029	0.0000	0.0040	0.7350	0.0040	0.2545
1	0.0039	0.0000	0.0050	0.7872	0.0049	0.2475
2	0.0058	0.0002	0.0130	0.9650	0.0106	0.1597
3	0.0052	0.0001	0.0084	0.6333	0.0081	0.3630
4	0.0061	0.0001	0.0118	0.9616	0.0094	0.2313
Mean	0.0048	0.0001	0.0084	0.8164	0.0074	0.2512
Std	0.0012	0.0001	0.0036	0.1298	0.0026	0.0653

```
In [15]: tuned_xgboost = tune_model(xgboost_model, choose_better=True)
```

	MAE	MSE	RMSE	R2	RMSLE	MAPE
Fold						
0	0.0031	0.0000	0.0035	0.8035	0.0034	0.2674
1	0.0038	0.0000	0.0045	0.8242	0.0044	0.2701
2	0.0131	0.0025	0.0501	0.4837	0.0400	0.2073
3	0.0045	0.0001	0.0078	0.6831	0.0075	0.2732
4	0.0037	0.0000	0.0051	0.9928	0.0050	0.2112
Mean	0.0056	0.0005	0.0142	0.7575	0.0121	0.2458
Std	0.0037	0.0010	0.0180	0.1688	0.0140	0.0300

Fitting 5 folds for each of 10 candidates, totalling 50 fits
Original model was better than the tuned model, hence it will be returned. NOT
E: The display metrics are for the tuned model (not the original one).

```
In [16]: from pycaret.regression import get_config
```

```
X_train = get_config('X_train')
X_train.head()
```

Out[16]:

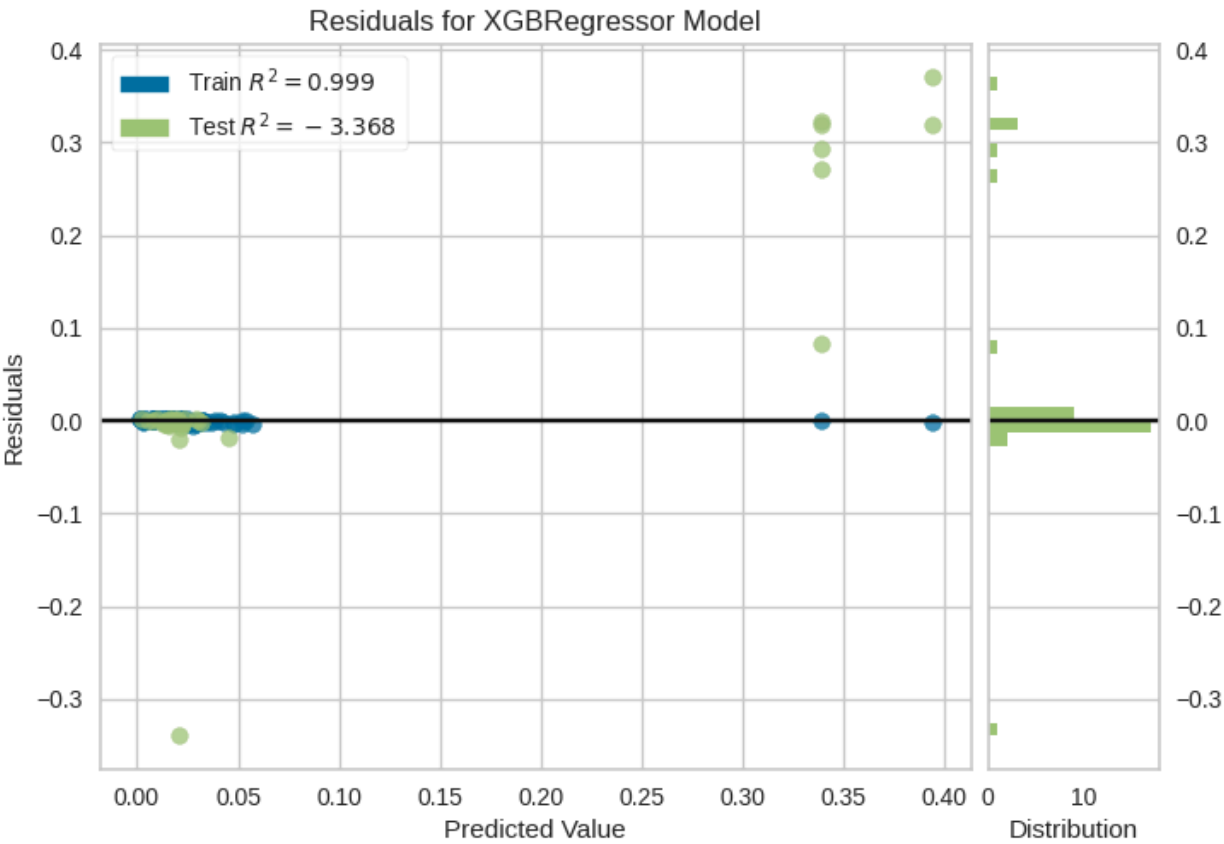
	year	aerial_distance_moscow_km
117	2023	2146.979980
65	2023	1608.829956
67	2025	1608.829956
31	2025	1777.410034
12	2022	1927.380005

```
In [17]: X_train_process = get_config('X_train_transformed')
X_train_process.head()
```

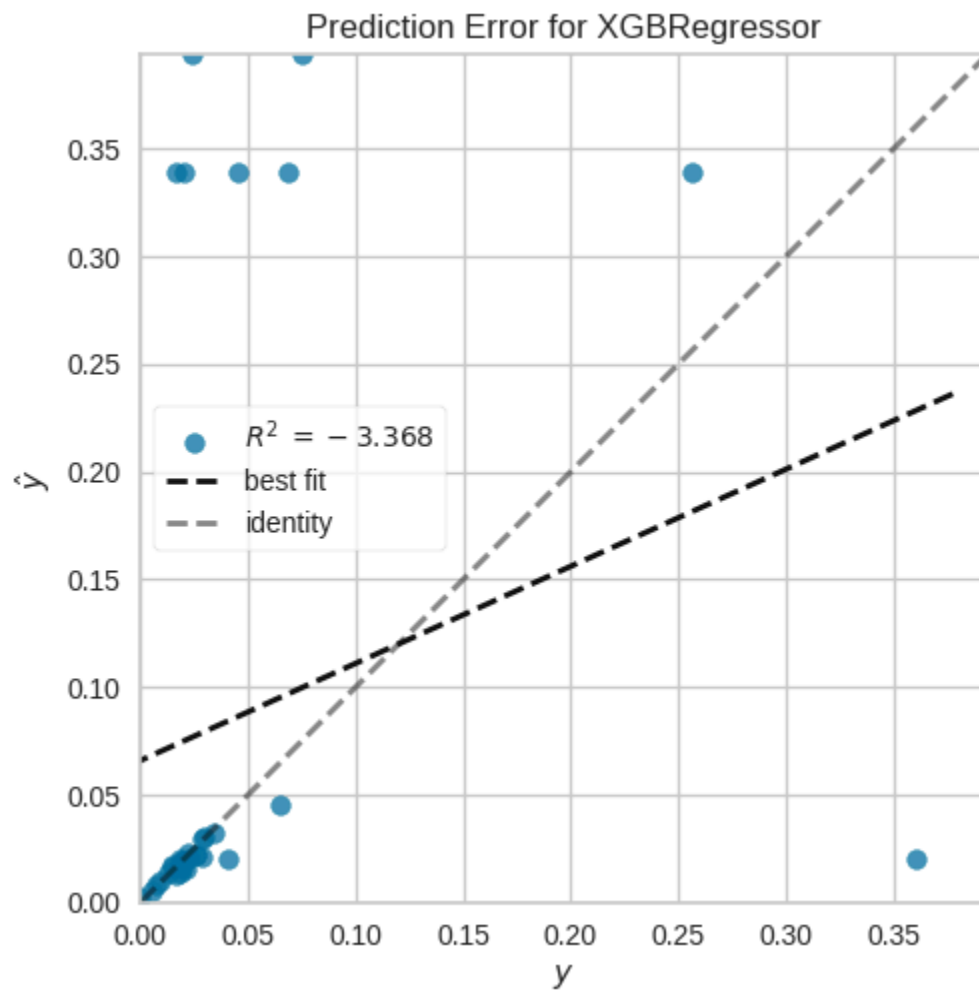
Out[17]:

	year_2023.0	year_2025.0	year_2022.0	year_2024.0	aerial_distance_moscow_km
117	1.834022	-0.609449	-0.566658	-0.588043	0.5
65	1.834022	-0.609449	-0.566658	-0.588043	-0.4
67	-0.545250	1.640825	-0.566658	-0.588043	-0.4
31	-0.545250	1.640825	-0.566658	-0.588043	-0.2
12	-0.545250	-0.609449	1.764734	-0.588043	0.0

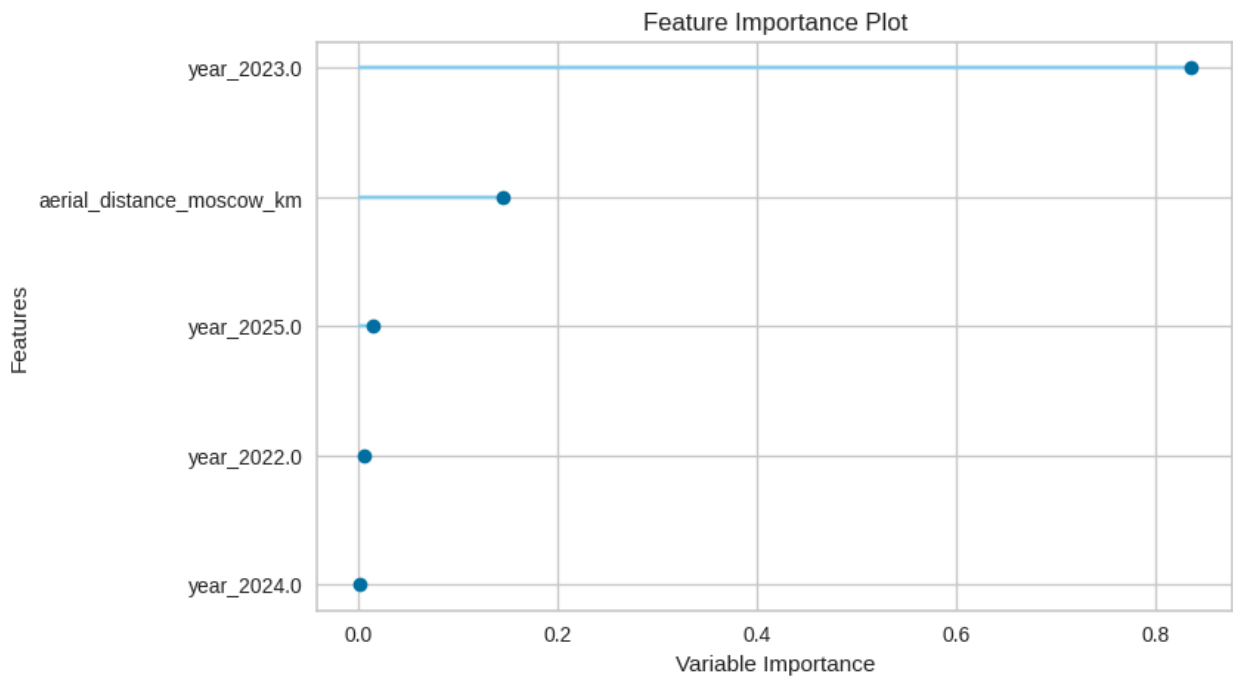
```
In [18]: plot_model(tuned_xgboost, plot='residuals')
```



```
In [19]: plot_model(tuned_xgboost, plot='error')
```



```
In [20]: plot_model(tuned_xgboost, plot='feature')
```



```
In [21]: evaluate_model(tuned_xgboost)
```

Following the February 2022 invasion of Ukraine, European governments braced for a rebalancing of the world order. German Chancellor Olaf Scholz deemed this moment to be Europe's *zeitenwende*—a historic turning point (Cepparulo & Pasimeni, 2024).

As governments create central budgets in the year prior based on geopolitical risks encountered recently, it can be derived that a massive and unpredicted reshuffling of risk in 2022 would be first noticed in most European government spending patterns in calendar year 2023. We see this global reshuffling play out precisely in our regression model. When using military expenditure as a share of GDP as our target, we are left with 2 features that are of the most importance: data from calendar year 2023 (> 80%) and the aerial distance that a European capital sits from Moscow (< 20%). With 4 years of war sampled in this dataset, (2022 through 2025,) it would be understandable that the year 2023 would bear the most importance, as it is the first year in which central budgets across Europe were given a chance to plan for and redirect capital toward growing uncertainty in the East.

To explain why our additional feature *aerial_distance_moscow_km* played significant importance, we need to explain the inspiration behind its inclusion in the dataset. Knezović and Tkalec (2025) posed an observation: For every 100 kilometers that a NATO member's capital is located further from Moscow, their average yearly military expenditure as a percentage of GDP has an associated

decrease of 0.02%. Knezović and Tkalec (2025) found that Western NATO members, protected by geography, distance, and buffer states, were much less likely to see the rise of Russian aggression as an immediate threat demanding significant defense budget changes.

Extreme Gradient Boosting proved to be the best performing regression model, with a MAE of 0.0048, MSE of 0.0001, a RMSE of 0.0084, and an R^2 Score of 0.8164. It is important to note that the target, *ME_spending_%_GDP*, is stored as a proportion: 0.03 meaning 3% of GDP. Therefore, an MAE of 0.0048 means approximately 0.48 percentage points, and not 0.0048 percentage points.

Extreme Gradient Boosting was uniquely positioned to excel where other regression techniques failed, as this was a fairly small dataset, (only 180 records,) and Extreme Gradient Boosting is known to handle interactions well (year * aerial distance from Moscow). Boosted tree structured models are better at modeling nonlinear relationships, as well as working with the necessary inclusion of outliers, (Ukraine and Russia,) whereas other regression models could easily be pulled astray.

7. Classification Modeling I: United States as Top Supplier Country

```
In [22]: from pycaret.classification import *
```

```
In [23]: classification_pipeline = setup(
    data=df,
    target='is_US_top_supplier_country?',
    session_id=42,
    normalize=True,
    ignore_features=['country', 'top_supplier_country', 'year'],
    numeric_imputation='mean',
    categorical_imputation='mode',
    fix_imbalance=True,
    fold=5
)
```

	Description	Value
0	Session id	42
1	Target is_US_top_supplier_country?	
2	Target type	Binary
3	Original data shape	(180, 19)
4	Transformed data shape	(221, 16)
5	Transformed train set shape	(166, 16)
6	Transformed test set shape	(55, 16)
7	Ignore features	3
8	Numeric features	15
9	Rows with missing values	70.0%
10	Preprocess	True
11	Imputation type	simple
12	Numeric imputation	mean
13	Categorical imputation	mode
14	Fix imbalance	True
15	Fix imbalance method	SMOTE
16	Normalize	True
17	Normalize method	zscore
18	Fold Generator	StratifiedKFold
19	Fold Number	5
20	CPU Jobs	-1
21	Use GPU	False
22	Log Experiment	False
23	Experiment Name	clf-default-name
24	USI	9f7b

```
In [24]: # Compare all models
best_model1 = compare_models()
```

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	Time (s)
dt	Decision Tree Classifier	0.7200	0.6853	0.5750	0.5741	0.5669	0.3629	0.3686	0.0000
et	Extra Trees Classifier	0.7200	0.7676	0.5778	0.6500	0.5753	0.3750	0.3997	0.0000
rf	Random Forest Classifier	0.7040	0.7700	0.4611	0.5444	0.4908	0.2960	0.2974	0.0000
qda	Quadratic Discriminant Analysis	0.7040	0.6208	0.4806	0.5686	0.5114	0.3062	0.3135	0.0000
gbc	Gradient Boosting Classifier	0.6880	0.7699	0.5528	0.5406	0.5339	0.3035	0.3144	0.0000
lightgbm	Light Gradient Boosting Machine	0.6880	0.7303	0.5056	0.5200	0.5032	0.2810	0.2859	0.0000
lr	Logistic Regression	0.6800	0.6699	0.6278	0.5319	0.5606	0.3195	0.3348	0.0000
ridge	Ridge Classifier	0.6720	0.6757	0.6306	0.5137	0.5482	0.3047	0.3219	0.0000
lda	Linear Discriminant Analysis	0.6720	0.6685	0.6556	0.5130	0.5587	0.3118	0.3326	0.0000
nb	Naive Bayes	0.6640	0.5984	0.3306	0.5190	0.3862	0.1748	0.1907	0.0000
dummy	Dummy Classifier	0.6640	0.5000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
svm	SVM - Linear Kernel	0.6560	0.6408	0.5861	0.5175	0.5061	0.2643	0.2947	0.0000
xgboost	Extreme Gradient Boosting	0.6400	0.6870	0.4806	0.4673	0.4598	0.1964	0.2055	0.0000
ada	Ada Boost Classifier	0.6240	0.6605	0.4111	0.4890	0.4383	0.1650	0.1665	0.0000
knn	K Neighbors Classifier	0.6160	0.7078	0.6306	0.4453	0.5055	0.2157	0.2382	0.0000

```
In [25]: et_model = create_model('et')
```

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
Fold							
0	0.8400	0.8676	0.7500	0.7500	0.7500	0.6324	0.6324
1	0.6800	0.7169	0.5000	0.5000	0.5000	0.2647	0.2647
2	0.6800	0.8162	0.7500	0.5000	0.6000	0.3506	0.3707
3	0.7600	0.8299	0.3333	1.0000	0.5000	0.3902	0.4924
4	0.6400	0.6076	0.5556	0.5000	0.5263	0.2373	0.2381
Mean	0.7200	0.7676	0.5778	0.6500	0.5753	0.3750	0.3997
Std	0.0716	0.0942	0.1585	0.2000	0.0947	0.1401	0.1469

```
In [26]: # Hyperparameter tuning

tuned_model1 = tune_model(et_model, optimize='AUC')
```

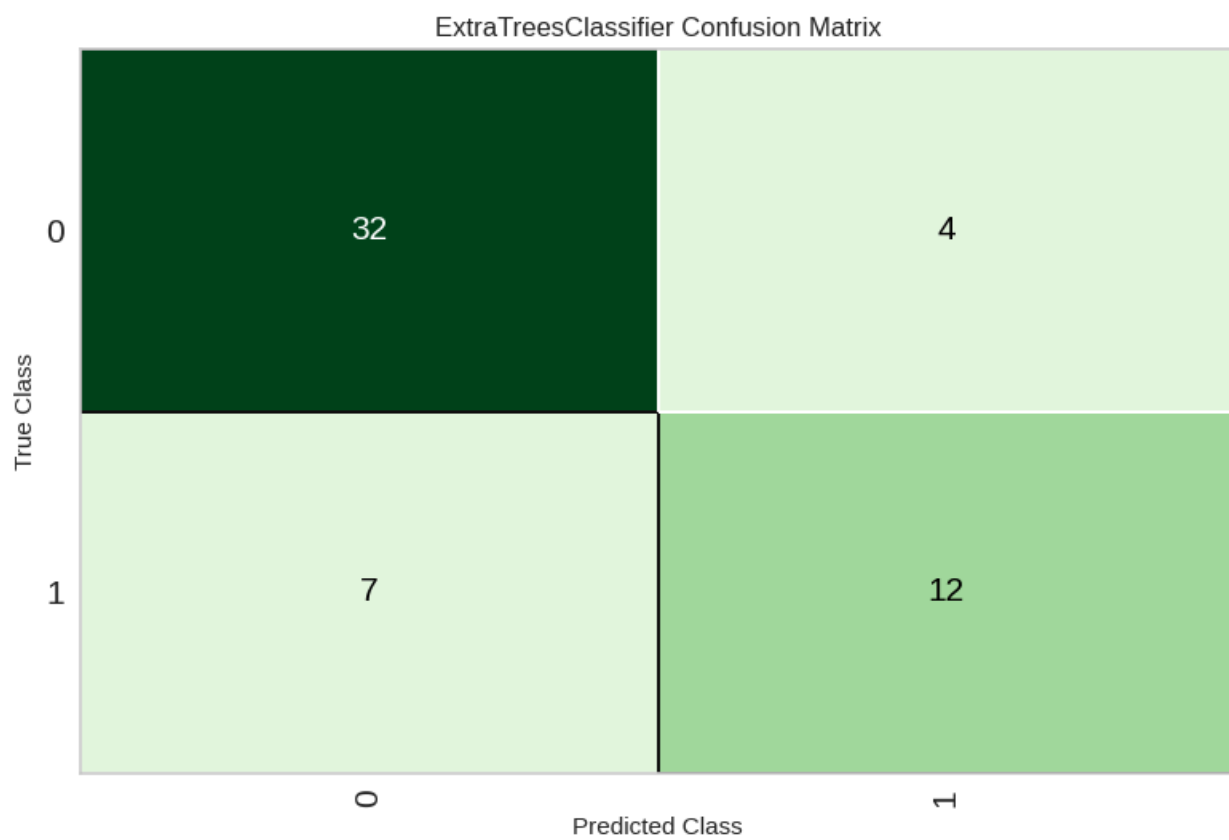
	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
Fold							
0	0.7600	0.8750	0.7500	0.6000	0.6667	0.4828	0.4901
1	0.7200	0.8309	0.8750	0.5385	0.6667	0.4479	0.4874
2	0.6400	0.8603	1.0000	0.4706	0.6400	0.3626	0.4706
3	0.6800	0.7500	0.3333	0.6000	0.4286	0.2308	0.2500
4	0.5600	0.6042	0.5556	0.4167	0.4762	0.1100	0.1134
Mean	0.6720	0.7841	0.7028	0.5251	0.5756	0.3268	0.3623
Std	0.0688	0.0998	0.2360	0.0723	0.1022	0.1389	0.1538

Fitting 5 folds for each of 10 candidates, totalling 50 fits

We're going to reject using this tuned model, as the reduction of accuracy and precision is too significant to justify the small increase in AUC.

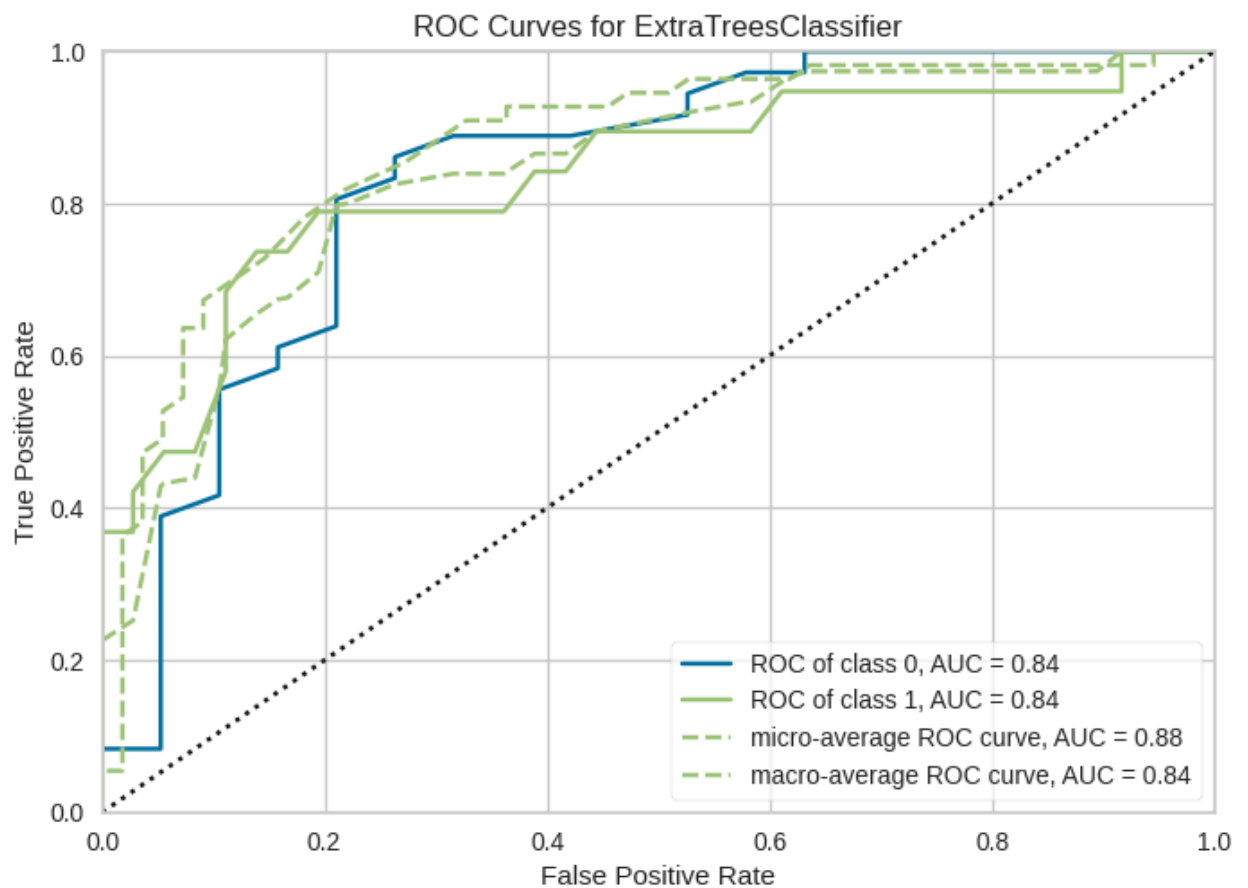
```
In [27]: # Confusion Matrix

plot_model(et_model, plot='confusion_matrix')
```



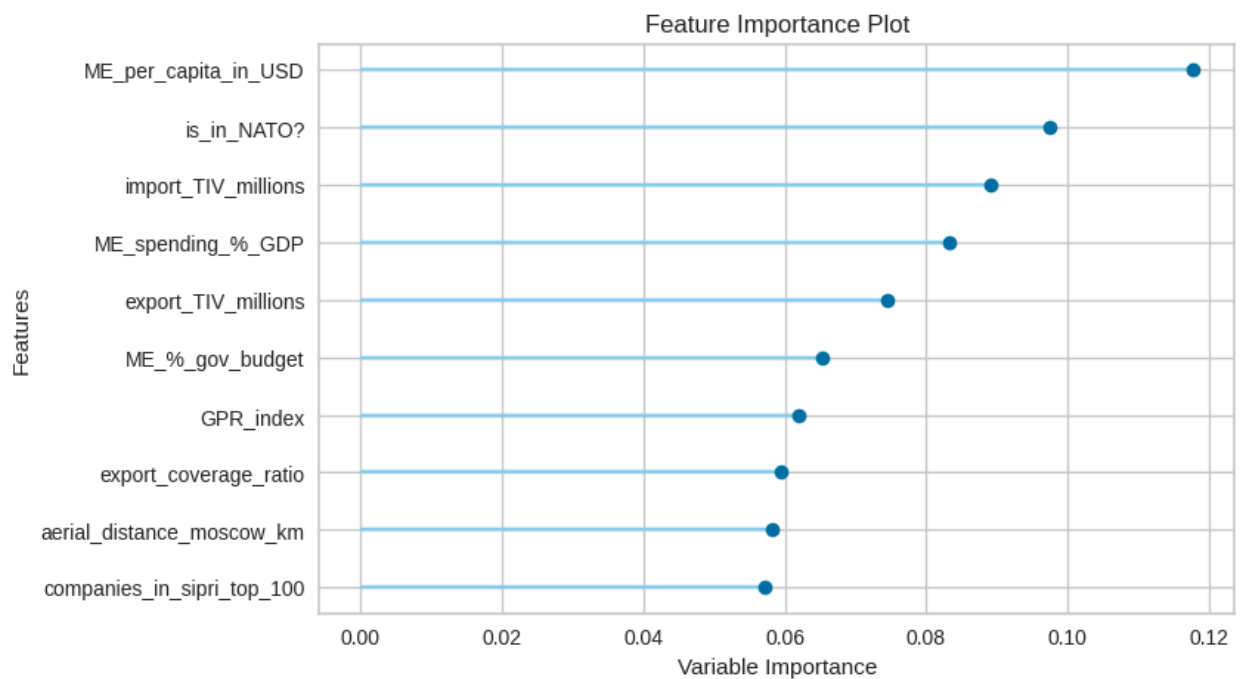
In [28]: *# ROC Curve*

```
plot_model(et_model, plot='auc')
```



```
In [29]: # Features that matter most

plot_model(et_model, plot='feature')
```



```
In [30]: # Interactive Evaluation Interface Dashboard
```

```
evaluate_model(et_model)
```

The first attempt at building a classification model involves a binary classification as to whether the United States is a country's top arms supplier.

The best performing model is an Extra Trees Classifier. Extra Trees creates random decision trees, then averages their predictions. For a nonlinear dataset that relies mostly on the importance of interactions, this ability to average decision tree predictions gives a leading edge in making a workable model of the most important interactions. Our Extra Trees Classifier held an accuracy of 0.72, an AUC of 0.77, a recall of 0.58, a precision of 0.65, and an F1 score of 0.58. When using pycaret's *tune_model()* function, we end up with a classifier with slightly higher AUC, but with dramatic reductions in accuracy and precision. For this reason, we reject using the tuned model produced and rely on the original Extra Trees model.

The original Extra Trees model manages to correctly classify 32 non-US top supplier country incidences and 12 US top supplier country incidences. We're left with 4 false-positive and 7 false-negative errors. Our most important features are Military Expenditure Per Capita (~0.119) and NATO membership (~0.097). It would make sense that NATO membership, as a US-led security alliance, would imply a strong likelihood of an American arms relationship. We can interpret our Feature Importance Plot as showing that this was in fact an interaction-heavy model.

8. Classification Modeling II: NATO Membership

```
In [31]: classification_pipeline2 = setup(  
    data=df,  
    target='is_in_NATO?',  
    session_id=42,  
    normalize=True,  
    ignore_features=['country', 'year'], # trying to prevent leakage  
    numeric_imputation='mean',  
    categorical_imputation='mode',  
    fix_imbalance=True,  
    fold=5  
)
```

	Description	Value
0	Session id	42
1	Target	is_in_NATO?
2	Target type	Binary
3	Original data shape	(180, 19)
4	Transformed data shape	(217, 39)
5	Transformed train set shape	(162, 39)
6	Transformed test set shape	(55, 39)
7	Ignore features	2
8	Numeric features	15
9	Categorical features	1
10	Rows with missing values	70.0%
11	Preprocess	True
12	Imputation type	simple
13	Numeric imputation	mean
14	Categorical imputation	mode
15	Maximum one-hot encoding	25
16	Encoding method	None
17	Fix imbalance	True
18	Fix imbalance method	SMOTE
19	Normalize	True
20	Normalize method	zscore
21	Fold Generator	StratifiedKFold
22	Fold Number	5
23	CPU Jobs	-1
24	Use GPU	False
25	Log Experiment	False
26	Experiment Name	clf-default-name
27	USI	fe39

```
In [32]: # Compare all models
best_model2 = compare_models()
```

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	Time (s)
ada	Ada Boost Classifier	0.9040	0.9458	0.8875	0.9589	0.9189	0.8026	0.8115	0.0000
gbc	Gradient Boosting Classifier	0.8880	0.9429	0.8882	0.9382	0.9097	0.7611	0.7685	0.0000
dt	Decision Tree Classifier	0.8800	0.8768	0.8897	0.9256	0.9060	0.7404	0.7453	0.0000
rf	Random Forest Classifier	0.8800	0.9104	0.9007	0.9122	0.9055	0.7403	0.7429	0.0000
xgboost	Extreme Gradient Boosting	0.8720	0.9235	0.8882	0.9088	0.8964	0.7279	0.7325	0.0000
lightgbm	Light Gradient Boosting Machine	0.8720	0.9125	0.8875	0.9107	0.8983	0.7254	0.7273	0.0000
et	Extra Trees Classifier	0.8640	0.8879	0.9125	0.8823	0.8956	0.6987	0.7054	0.0000
qda	Quadratic Discriminant Analysis	0.6880	0.7934	0.6044	0.8842	0.7136	0.3920	0.4313	0.0000
lr	Logistic Regression	0.6800	0.6960	0.7022	0.7831	0.7355	0.3250	0.3333	0.0000
knn	K Neighbors Classifier	0.6800	0.7137	0.7287	0.7687	0.7466	0.3103	0.3136	0.0000
svm	SVM - Linear Kernel	0.6800	0.7056	0.7162	0.7830	0.7423	0.3137	0.3253	0.0000
lda	Linear Discriminant Analysis	0.6640	0.6592	0.7147	0.7565	0.7287	0.2768	0.2858	0.0000
ridge	Ridge Classifier	0.6560	0.6618	0.7022	0.7527	0.7194	0.2642	0.2738	0.0000
nb	Naive Bayes	0.5120	0.7817	0.2949	0.8529	0.4351	0.1598	0.2332	0.0000
dummy	Dummy Classifier	0.3520	0.5000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

```
In [33]: ada_model = create_model('ada')
```

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
Fold							
0	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1	0.9200	0.9861	0.9375	0.9375	0.9375	0.8264	0.8264
2	0.7600	0.8542	0.7500	0.8571	0.8000	0.5033	0.5104
3	0.8400	0.8889	0.7500	1.0000	0.8571	0.6835	0.7206
4	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Mean	0.9040	0.9458	0.8875	0.9589	0.9189	0.8026	0.8115
Std	0.0933	0.0619	0.1146	0.0564	0.0793	0.1909	0.1845

```
In [34]: # Hyperparameter tuning

tuned_model2 = tune_model(ada_model, optimize='AUC')
```

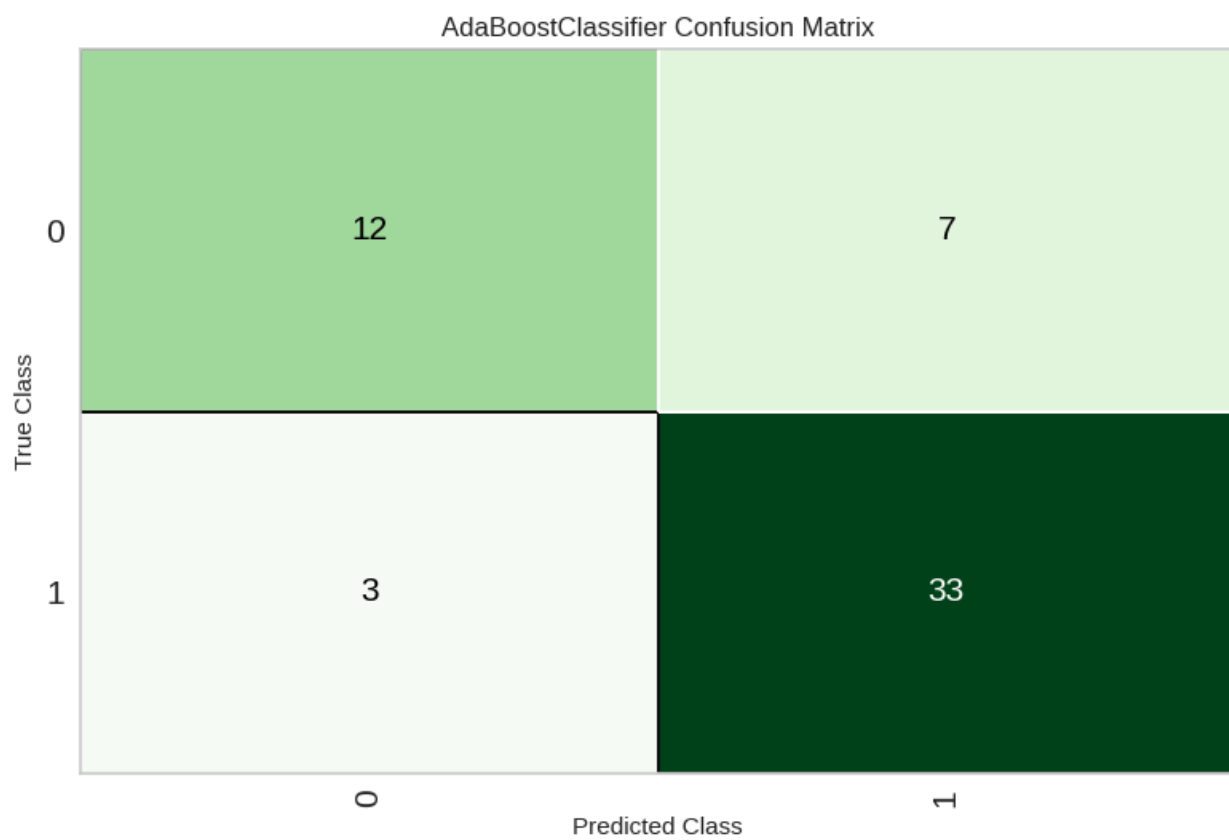
	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
Fold							
0	0.9600	0.9861	1.0000	0.9412	0.9697	0.9110	0.9147
1	0.8800	0.9306	0.9375	0.8824	0.9091	0.7331	0.7360
2	0.8000	0.8507	0.9375	0.7895	0.8571	0.5318	0.5541
3	0.8000	0.8472	0.7500	0.9231	0.8276	0.5955	0.6138
4	0.9600	0.9926	1.0000	0.9444	0.9714	0.9049	0.9091
Mean	0.8800	0.9214	0.9250	0.8961	0.9070	0.7353	0.7455
Std	0.0716	0.0630	0.0919	0.0577	0.0581	0.1553	0.1479

Fitting 5 folds for each of 10 candidates, totalling 50 fits
Original model was better than the tuned model, hence it will be returned. NOT
E: The display metrics are for the tuned model (not the original one).

Pycaret's tune_model() returns our original model, as the tuned models did not perform better.

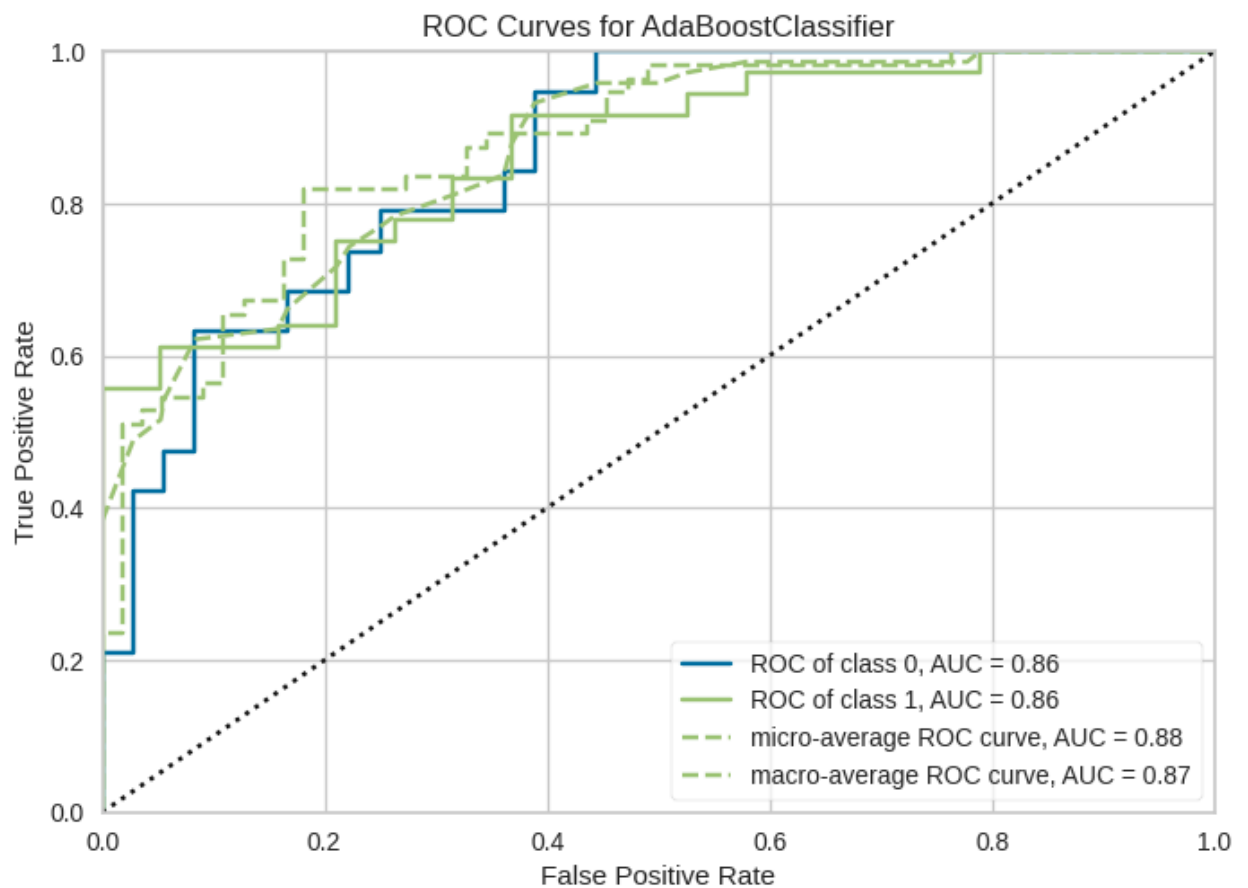
```
In [35]: # Confusion Matrix

plot_model(tuned_model2, plot='confusion_matrix')
```



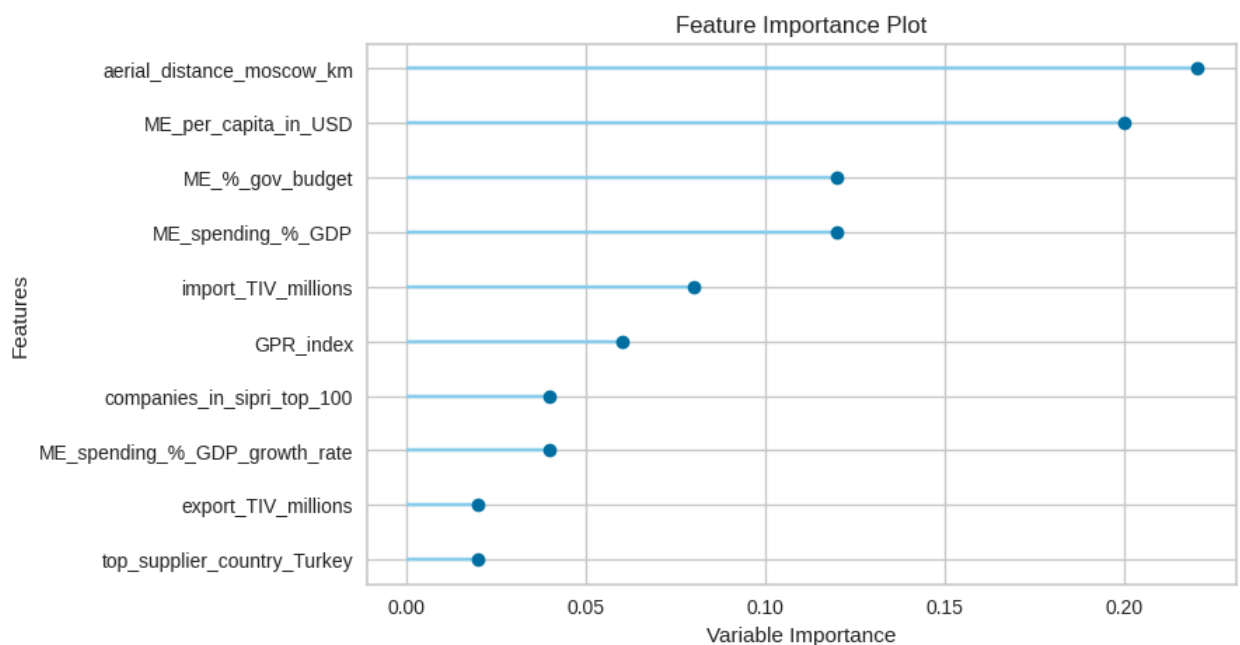
In [36]: *# ROC Curve*

```
plot_model(tuned_model2, plot='auc')
```



```
In [37]: # Features that matter most

plot_model(tuned_model2, plot='feature')
```



```
In [38]: # Interactive Evaluation Interface Dashboard
```

```
evaluate_model(tuned_model2)
```

The second attempt at building a classification model involves a binary classification as to whether a country is a member of NATO. One interesting caveat in this target data is that Finland became a NATO member in 2023, and Sweden a member in 2024.

The best performing model for this task was an AdaBoost Classifier. In contrast to other tree models like Extra Trees, AdaBoost builds decision trees in a sequence, having each new tree correct earlier classification mistakes. Our AdaBoost Classifier held an accuracy of 0.90, an AUC of 0.95, a recall of 0.89, a precision of 0.96, and an F1 score of 0.92. Across the board, this classifier performed better than our US Top Supplier Classifier on every metric. When using `pycaret's tune_model()` function, we didn't see any improvement over our original model.

The AdaBoost Classifier managed to correctly classify 12 non-NATO member incidences and 33 NATO member incidences. We're left with 7 false-positive and 3 false-negative errors. Our most important features are Aerial Distance From Moscow (~0.22) and Military Expenditure Per Capita (0.20). It makes sense that distance from Moscow would correlate highly with NATO membership, as NATO was a Western alliance that slowly moved east. Many countries closer to Moscow (Georgia, Ukraine, Moldova etc.) are not members, as Russia has prevented their acceptance into the alliance by claiming territory within their borders. NATO members also have expected military spending targets, so it would make sense that a high military expenditure per capita would correlate with NATO membership.

Decision Tree models usually come out on top when we pit several classification models against each other, but we can infer that the reason AdaBoost triumphed over other decision tree models was that the dataset had 2 very important features that together held a variable importance of around 0.42. With 2 features creating strong thresholds, AdaBoost's sequential tree design was uniquely suited to pick up on small additional observations missed that were not geographical (aerial distance from Moscow) or financial (military expenditure). To put it simply, all decision tree models picked up on `aerial_distance_moscow_km` and `ME_per_capita_in_USD` being the most important features, but AdaBoost was able to find minor tweaks for other features once it understood the major importance of the first two.

9. Clustering and Dimensionality Reduction: Finding Commonalities

```
In [39]: from pycaret.clustering import *
```

```
In [40]: from scipy.cluster.hierarchy import dendrogram, linkage

# This function helps create all future dendrogram visualizations.

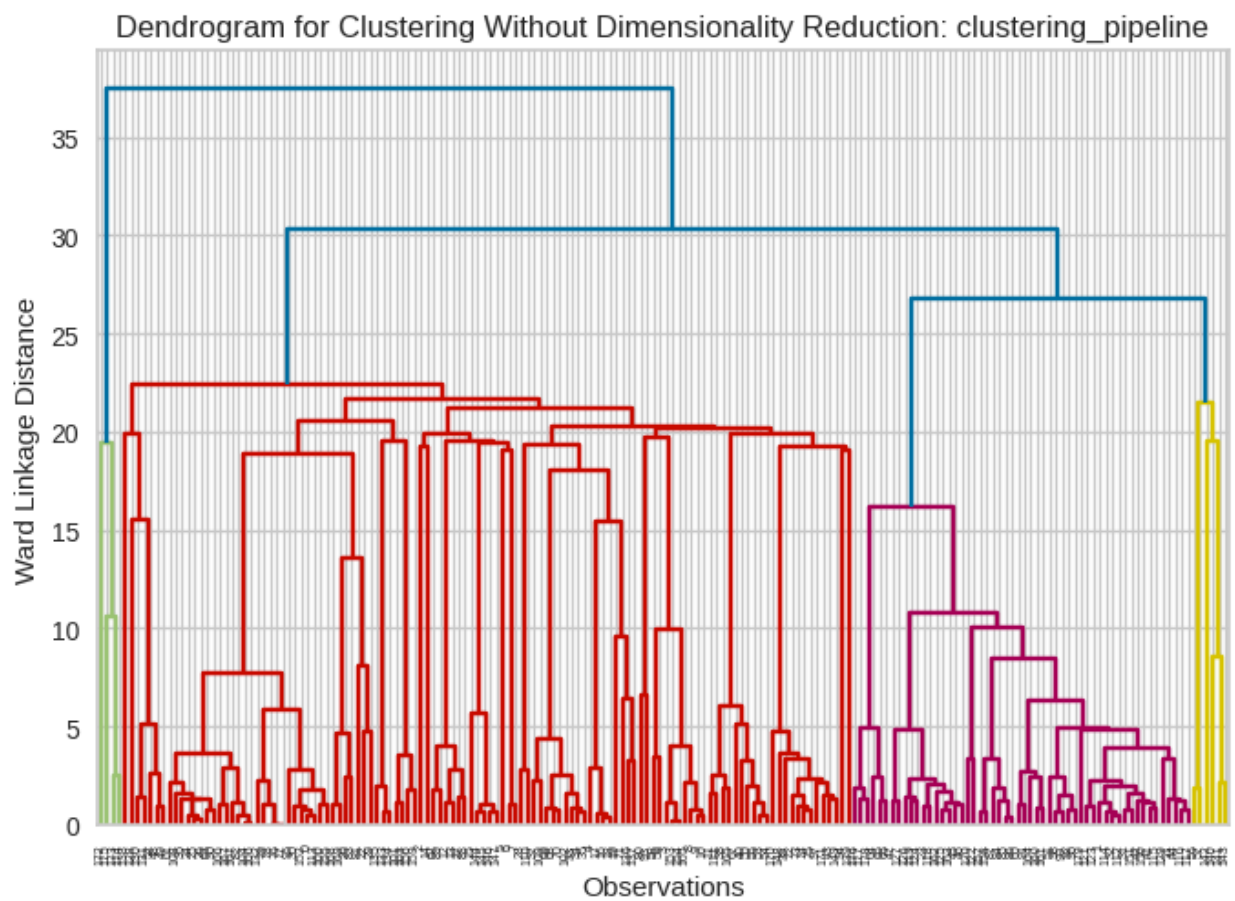
def dendro(title):
    dendrogram(linkage(get_config('X_transformed'), method='ward'))
    plt.xlabel('Observations')
    plt.ylabel('Ward Linkage Distance')
    plt.title(title)
    plt.show()
```

9.1 Clustering Without Dimensionality Reduction

```
In [41]: clustering_pipeline = setup(
    data=df,
    ignore_features=[
        'country',
        'year'
    ],
    numeric_imputation='mean',
    categorical_imputation='mode',
    normalize=True,
    normalize_method='zscore',
    session_id=42
)
```

	Description	Value
0	Session id	42
1	Original data shape	(180, 19)
2	Transformed data shape	(180, 41)
3	Ignore features	2
4	Numeric features	16
5	Categorical features	1
6	Rows with missing values	70.0%
7	Preprocess	True
8	Imputation type	simple
9	Numeric imputation	mean
10	Categorical imputation	mode
11	Maximum one-hot encoding	-1
12	Encoding method	None
13	Normalize	True
14	Normalize method	zscore
15	CPU Jobs	-1
16	Use GPU	False
17	Log Experiment	False
18	Experiment Name	cluster-default-name
19	USI	a0b7

```
In [42]: dendro("Dendrogram for Clustering Without Dimensionality "
               "Reduction: clustering_pipeline")
```



This looks atrocious compared to the results we'll get when we use dimensionality reduction.

```
In [43]: print(models())
```

ID	Name \
kmeans	K-Means Clustering
ap	Affinity Propagation
meanshift	Mean Shift Clustering
sc	Spectral Clustering
hclust	Agglomerative Clustering
dbscan	Density-Based Spatial Clustering
optics	OPTICS Clustering
birch	Birch Clustering

ID	Reference
kmeans	sklearn.cluster._kmeans.KMeans
ap	sklearn.cluster._affinity_propagation.Affinity...
meanshift	sklearn.cluster._mean_shift.MeanShift
sc	sklearn.cluster._spectral.SpectralClustering
hclust	sklearn.cluster._agglomerative.AgglomerativeCl...
dbscan	sklearn.cluster._dbscan.DBSCAN
optics	sklearn.cluster._optics.OPTICS
birch	sklearn.cluster._birch.Birch

```
In [44]: kmeans_model = create_model('kmeans', num_clusters=4)
```

	Silhouette	Calinski-Harabasz	Davies-Bouldin	Homogeneity	Rand Index	Completeness
0	0.2167	11.6329	1.1496	0	0	0

```
In [45]: hclust_model = create_model('hclust', num_clusters=6)
```

	Silhouette	Calinski-Harabasz	Davies-Bouldin	Homogeneity	Rand Index	Completeness
0	0.1231	13.0067	2.3107	0	0	0

The first attempt at clustering does not involve dimensionality reduction. It produced results that were not significant or meaningful. The dendrogram plot is unreadable, and we're left with our best silhouette score being only 0.22 (a k-means clustering model).

9.2 Linear PCA + Clustering

```
In [46]: clustering_pipeline2 = setup(
    data=df,
    ignore_features=[
        'country',
        'year'
    ],
    numeric_imputation='mean',
    categorical_imputation='mode',
```

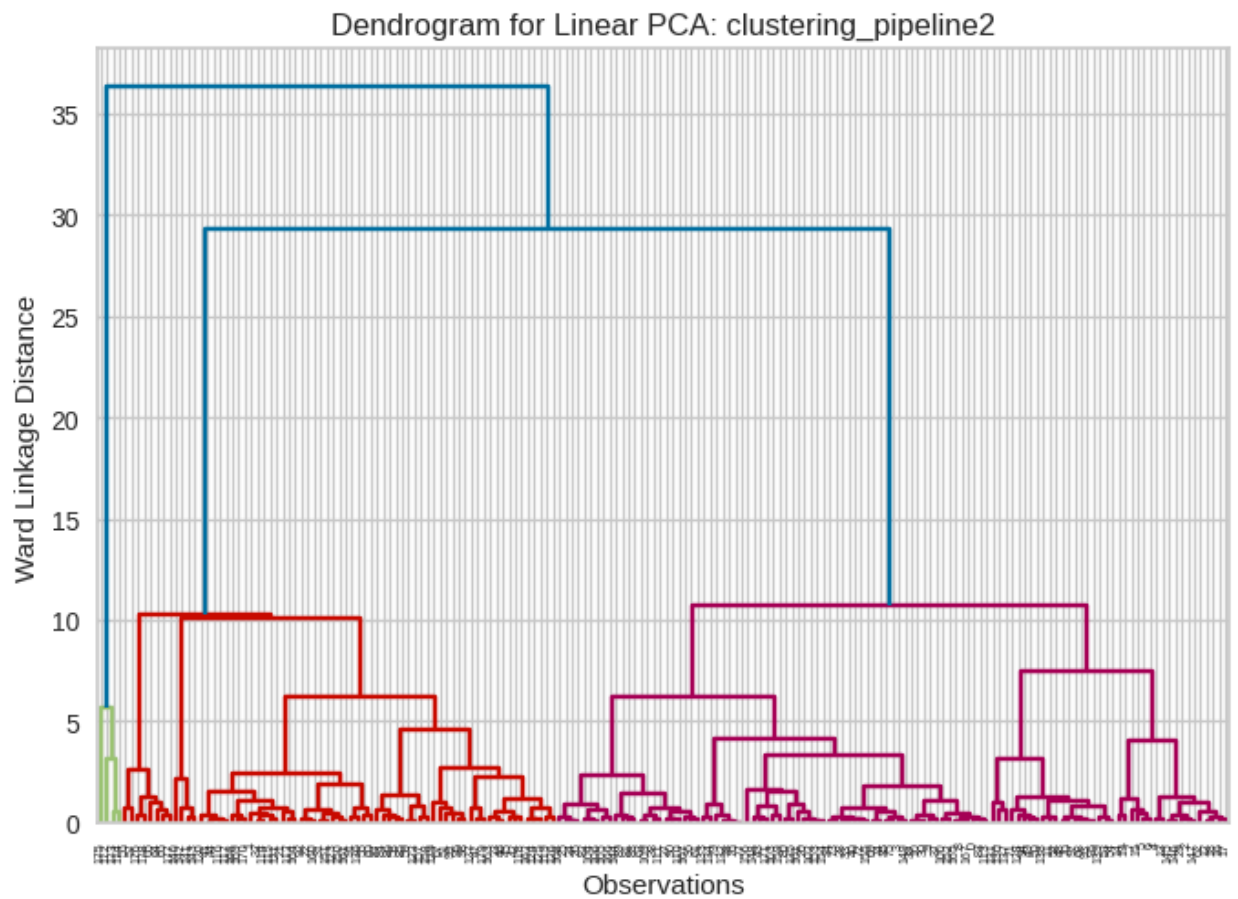
```

normalize=True,
normalize_method='zscore',
session_id=42,
pca=True,
pca_method='linear',
pca_components=2
)

```

	Description	Value
0	Session id	42
1	Original data shape	(180, 19)
2	Transformed data shape	(180, 2)
3	Ignore features	2
4	Numeric features	16
5	Categorical features	1
6	Rows with missing values	70.0%
7	Preprocess	True
8	Imputation type	simple
9	Numeric imputation	mean
10	Categorical imputation	mode
11	Maximum one-hot encoding	-1
12	Encoding method	None
13	Normalize	True
14	Normalize method	zscore
15	PCA	True
16	PCA method	linear
17	PCA components	2
18	CPU Jobs	-1
19	Use GPU	False
20	Log Experiment	False
21	Experiment Name	cluster-default-name
22	USI	dc51

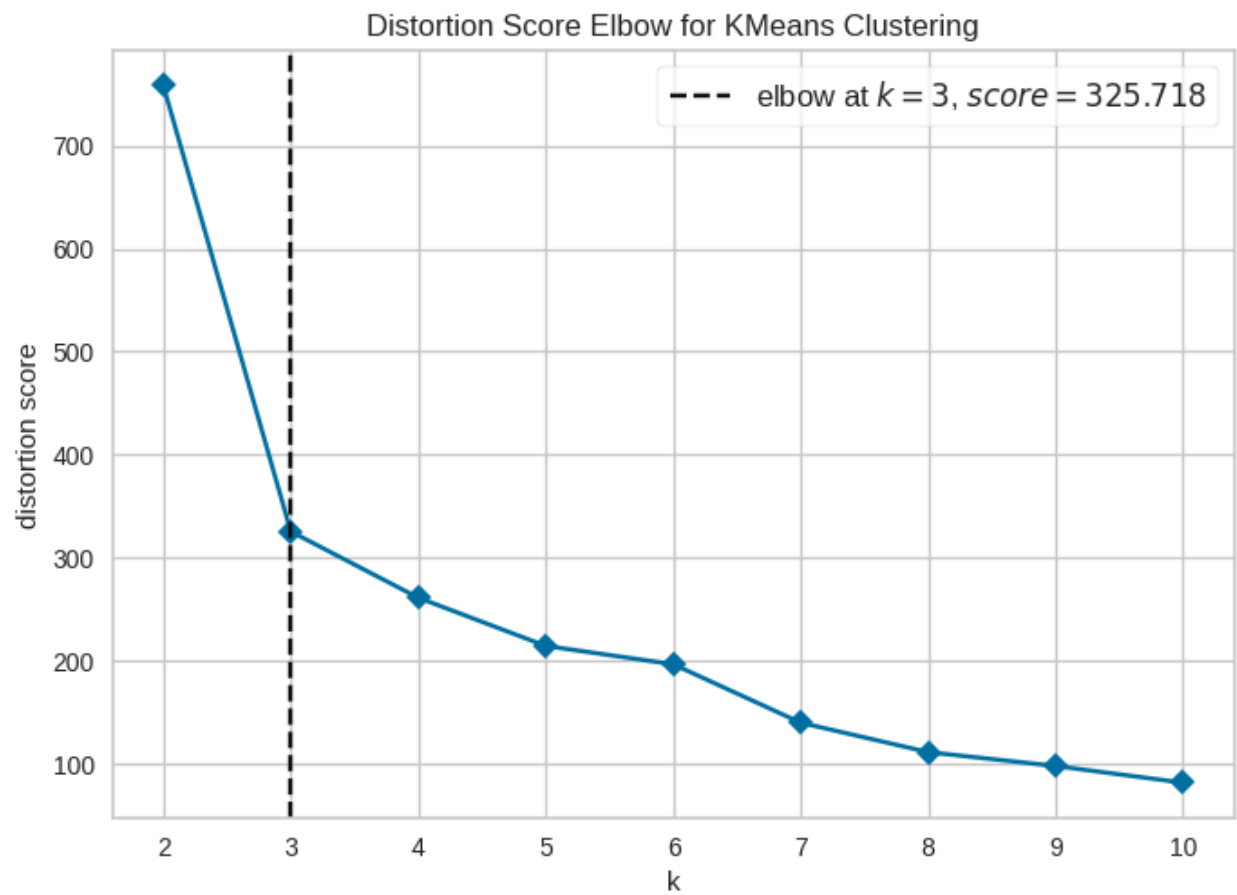
```
In [47]: dendro("Dendrogram for Linear PCA: clustering_pipeline2")
```



```
In [48]: kmeans_model2 = create_model('kmeans', num_clusters=3)
```

	Silhouette	Calinski-Harabasz	Davies-Bouldin	Homogeneity	Rand Index	Completeness
0	0.5384	297.2208	0.5493	0	0	0

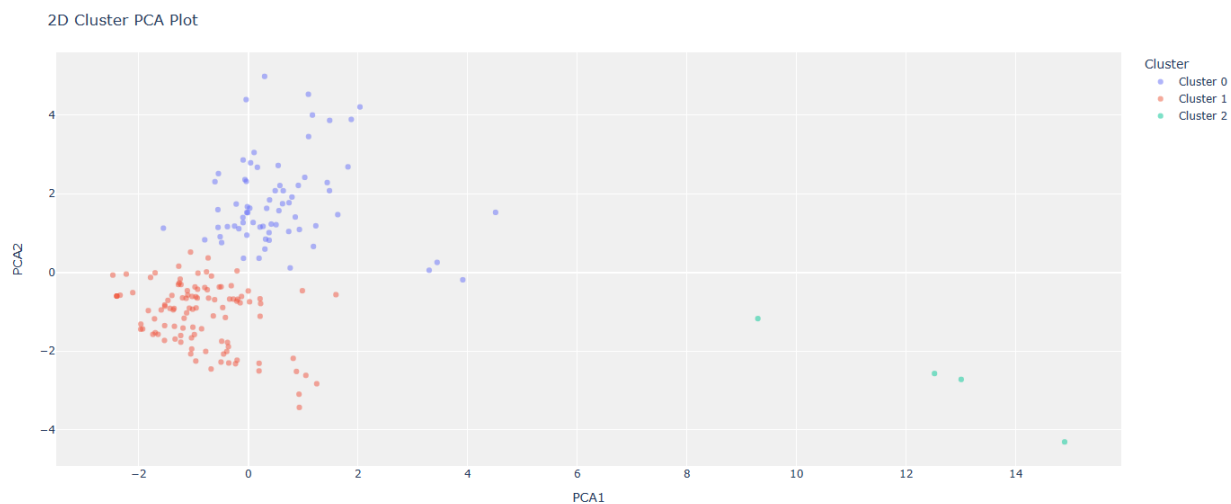
```
In [49]: plot_model(kmeans_model2, plot='elbow')
```



```
In [50]: plot_model(kmeans_model2, plot='cluster')
```

```
In [51]: # The above cluster plot has trouble rendering correctly in most notebooks.  
# Here's a .png file of what it looks like.
```

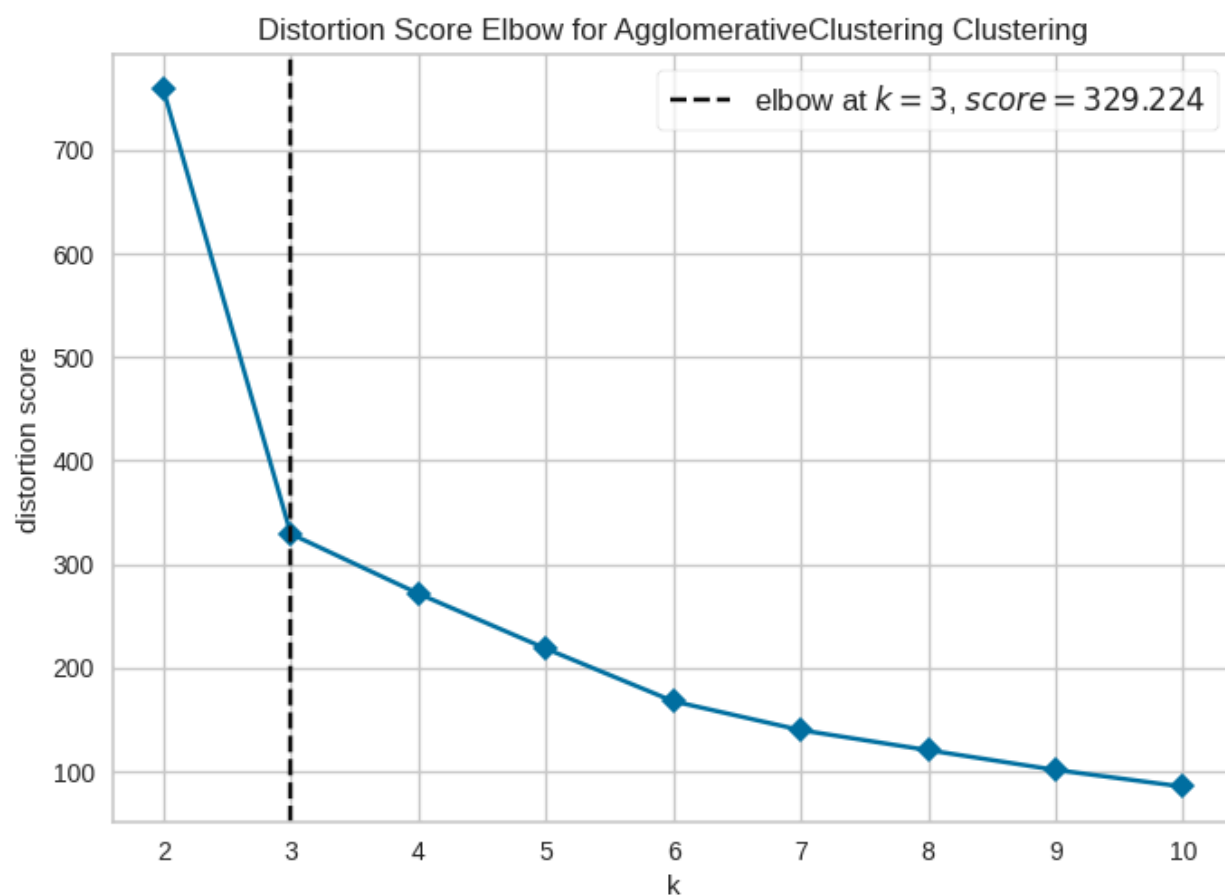
```
display(Image(filename="linear_PCA_kmeans.png", width=1100))
```



```
In [52]: hclust_model2 = create_model('hclust', num_clusters=3)
```

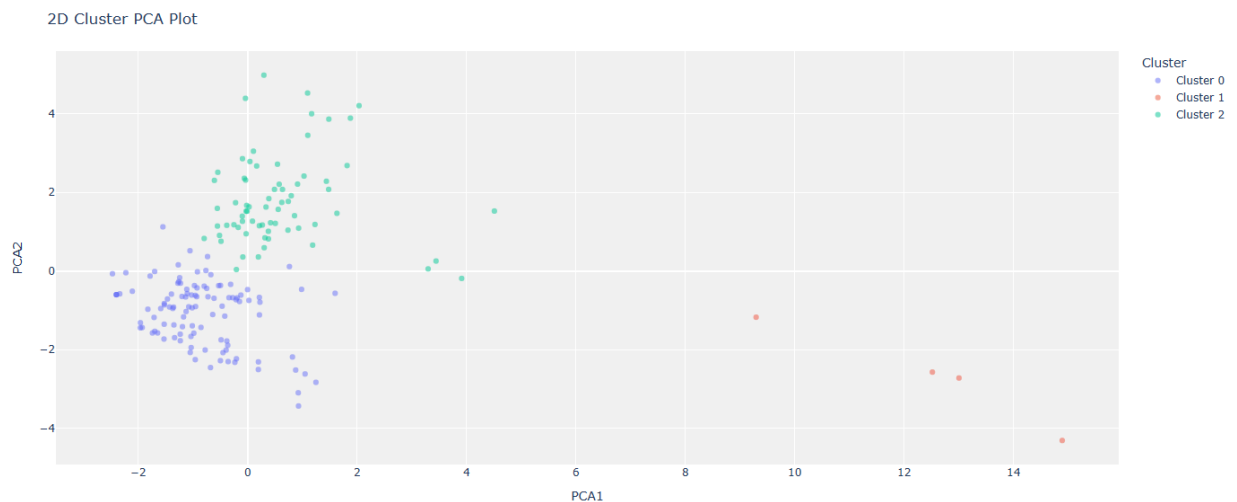
	Silhouette	Calinski-Harabasz	Davies-Bouldin	Homogeneity	Rand Index	Completeness
0	0.5327	293.1130	0.5519	0	0	0

```
In [53]: plot_model(hclust_model2, plot='elbow')
```



```
In [54]: plot_model(hclust_model2, plot='cluster')
```

```
In [55]: # The above cluster plot has trouble rendering correctly in most notebooks.  
# Here's a .png file of what it looks like.  
display(Image(filename="linear_PCA_hclust.png", width=1100))
```



The second clustering attempt uses linear PCA as its method of dimensionality reduction with improved results. When reducing our features down to two linear principal components, we get a dendrogram and distortion score elbow plot that favors 3 clusters (in both our k-means clustering model and our hierarchical agglomerative clustering model). When creating 2D scatter plots of the clusters, our k-means model and our hierarchical agglomerative model give us nearly identical results, with the only difference being small border data points between the top-left corner and bottom-left corner clusters. The top-left corner cluster groups together countries known to be major military import or export players: Russia, France, Turkey, United Kingdom, Poland, Italy, and Germany. The bottom-left corner cluster groups together countries that have relatively low military spending needs: Ireland, Iceland, Malta, Kosovo, etc. Ukraine, (a strong outlier as an active war zone,) becomes its own cluster. The k-means model gives us the slightly better of the two silhouette scores at 0.54.

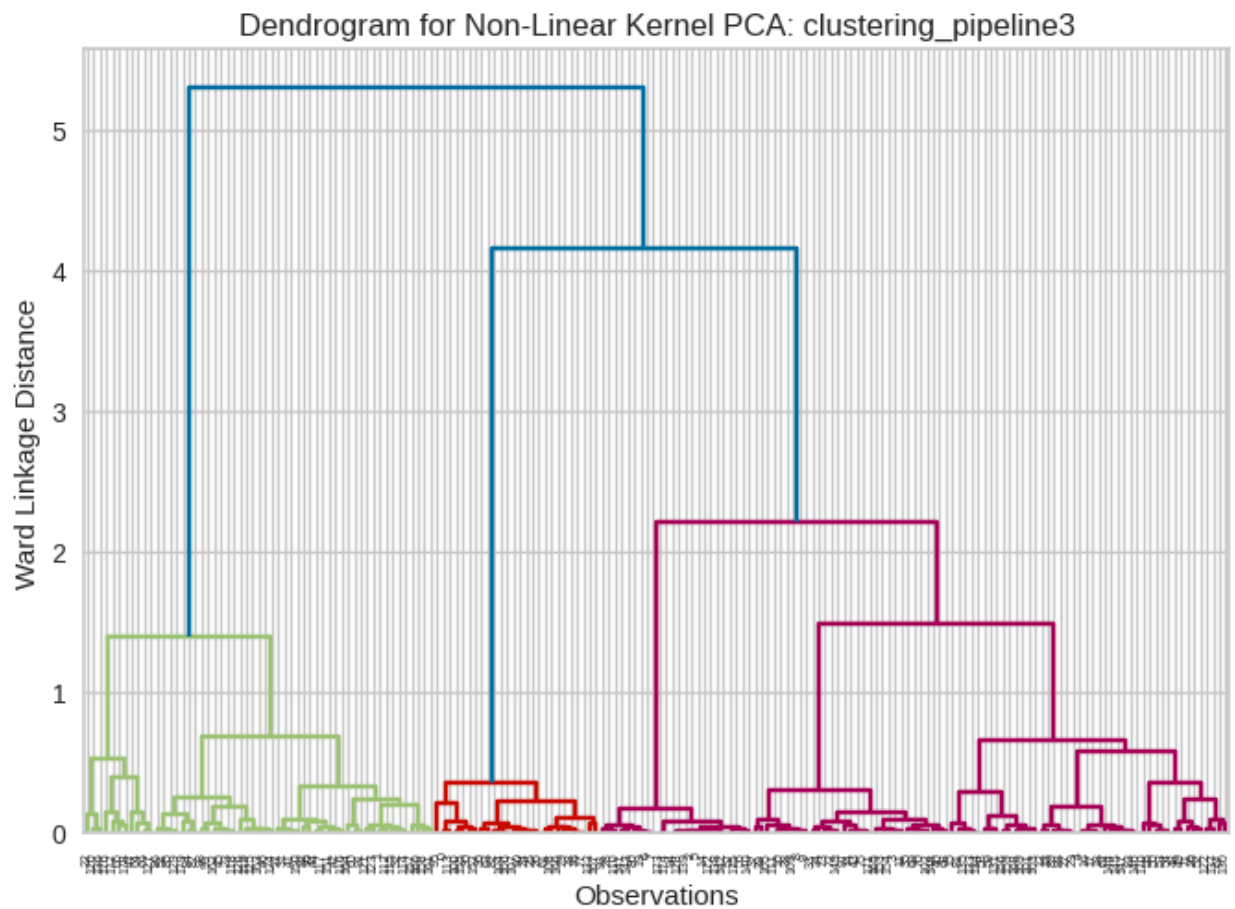
9.3 Nonlinear Kernel PCA + Clustering

```
In [56]: clustering_pipeline3 = setup(
    data=df,
    ignore_features=[
        'country',
        'year'
    ],
    numeric_imputation='mean',
    categorical_imputation='mode',
    normalize=True,
    normalize_method='zscore',
    session_id=42,
    pca=True,
    pca_method='kernel',
    pca_components=2
```

)

	Description	Value
0	Session id	42
1	Original data shape	(180, 19)
2	Transformed data shape	(180, 2)
3	Ignore features	2
4	Numeric features	16
5	Categorical features	1
6	Rows with missing values	70.0%
7	Preprocess	True
8	Imputation type	simple
9	Numeric imputation	mean
10	Categorical imputation	mode
11	Maximum one-hot encoding	-1
12	Encoding method	None
13	Normalize	True
14	Normalize method	zscore
15	PCA	True
16	PCA method	kernel
17	PCA components	2
18	CPU Jobs	-1
19	Use GPU	False
20	Log Experiment	False
21	Experiment Name	cluster-default-name
22	USI	6940

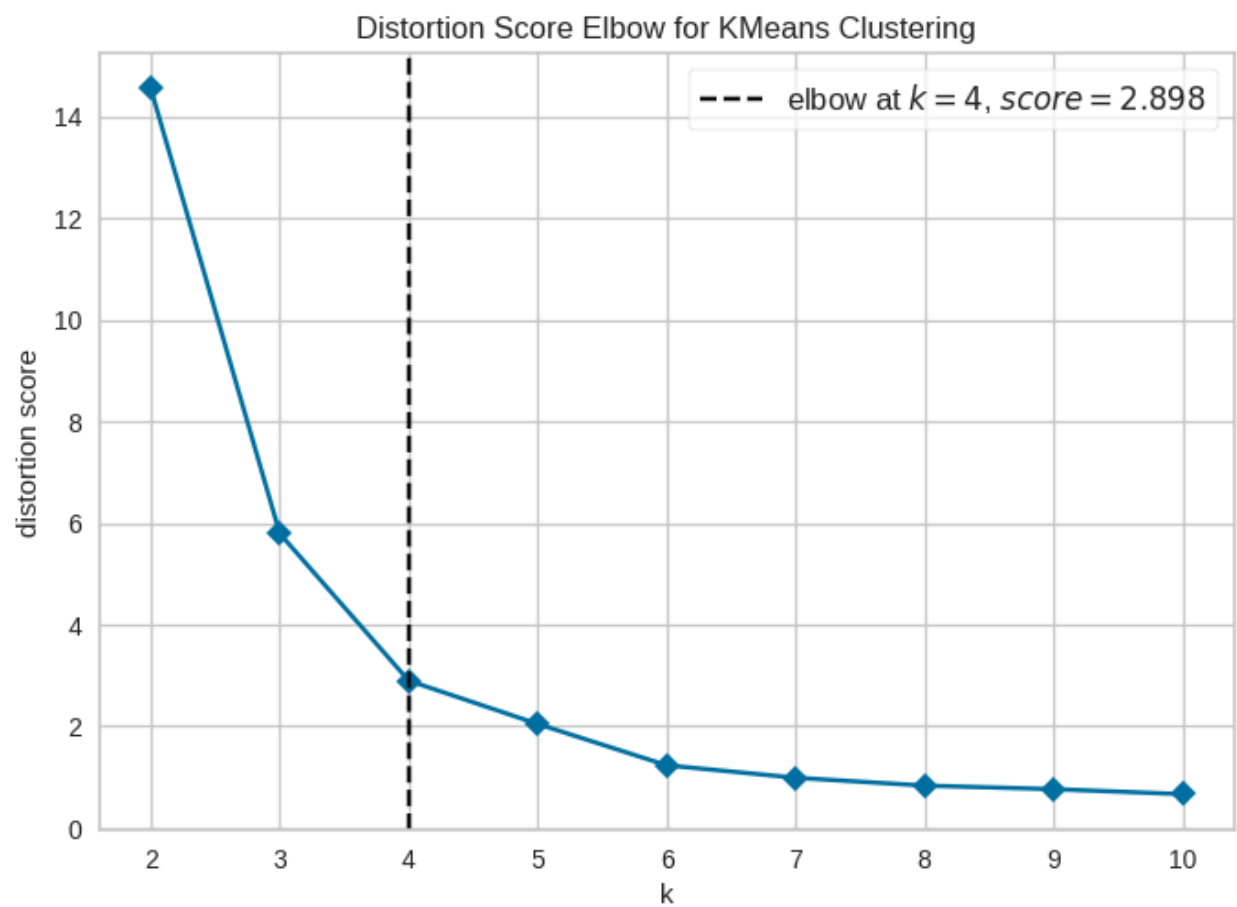
```
In [57]: dendro("Dendrogram for Non-Linear Kernel PCA: clustering_pipeline3")
```



```
In [58]: kmeans_model3 = create_model('kmeans', num_clusters=3)
```

	Silhouette	Calinski-Harabasz	Davies-Bouldin	Homogeneity	Rand Index	Completeness
0	0.6504	349.3418	0.4634	0	0	0

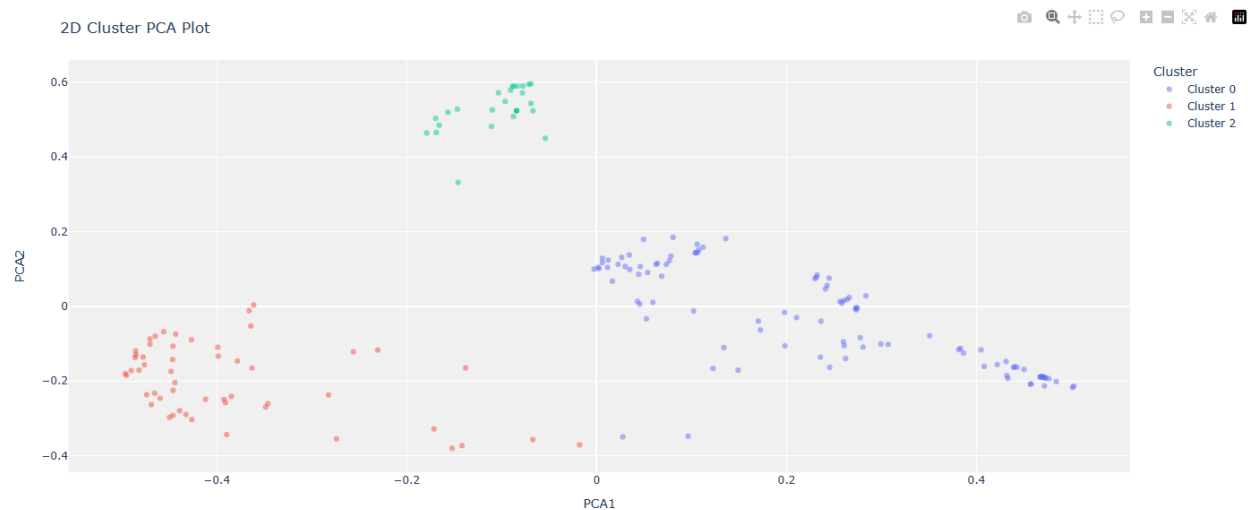
```
In [59]: plot_model(kmeans_model3, plot='elbow')
```



```
In [60]: plot_model(kmeans_model3, plot='cluster')
```

In [61]: *# The above cluster plot has trouble rendering correctly in most notebooks.
Here's a .png file of what it looks like.*

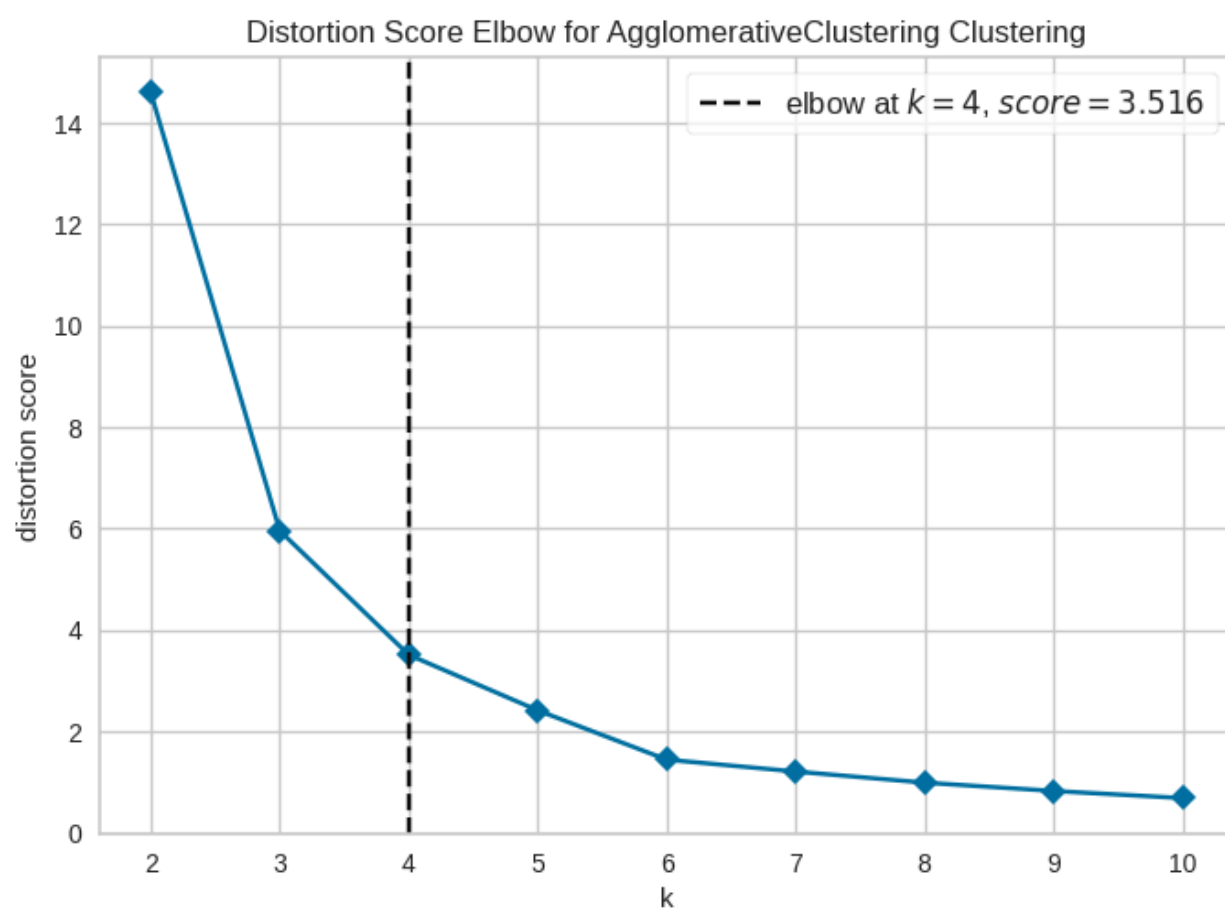
```
display(Image(filename="nonlinear_PCA_kmeans.png", width=1100))
```



```
In [62]: hclust_model3 = create_model('hclust', num_clusters=3)
```

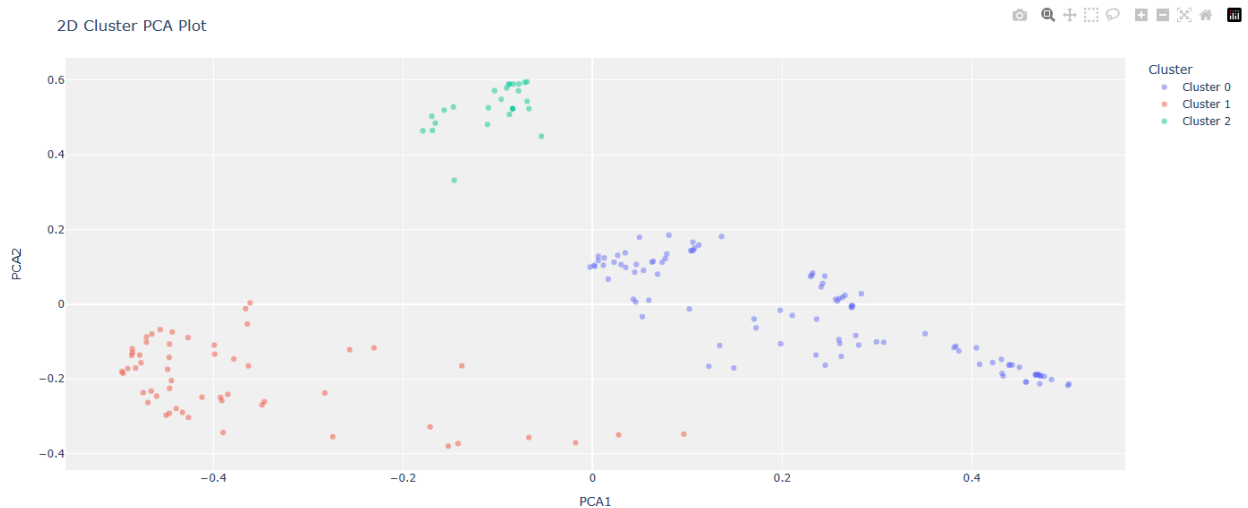
	Silhouette	Calinski-Harabasz	Davies-Bouldin	Homogeneity	Rand Index	Completeness
0	0.6450	337.0767	0.4802	0	0	0

```
In [63]: plot_model(hclust_model3, plot='elbow')
```



```
In [64]: plot_model(hclust_model3, plot='cluster')
```

```
In [65]: # The above cluster plot has trouble rendering correctly in most notebooks.  
# Here's a .png file of what it looks like.  
  
display(Image(filename="nonlinear_PCA_hclust.png", width=1100))
```



The third clustering attempt uses nonlinear kernel PCA as its method of dimensionality reduction. When reducing our features down to two non-linear principal components, we get a dendrogram that favors 3 clusters (in both our k-means model and our hierarchical agglomerative model). When creating 2D scatter plots of the clusters, our k-means model and our agglomerative model once again give us nearly identical results (the main difference being a few United Kingdom data points at the bottom center of both plots).

The top-center of the scatter plot gives us our tightest cluster, composed mostly of smaller countries without much need in terms of military spending: Iceland, Portugal, Malta, Montenegro, etc. Many of these countries have periods of low or no arms imports. The bottom-right cluster includes countries with major military capabilities: Russia, Ukraine, France, Poland, Turkey) but many additional countries that don't form clear similarities. In the bottom-left corner, we get a cluster of many NATO members: (Norway, Sweden, Lithuania, Netherlands, Slovakia, Germany, etc.) Another commonality of this cluster is that most of the countries have the US as their top arms supplier. Both the k-means model and the agglomerative model give us our best resulting silhouette score: 0.65.

It should be noted that Ukraine's data points were a massive outlier. When forcing the number of clusters to be exactly 2, all 4 of the plotted models preferred clustering Ukraine on its own, with the remaining cluster containing all other European countries. Ukraine is the main war front, having transformed its economy into an engine to sustain a long-term, grinding war of attrition.

10. References

Bachtiar, M., Ahmad, I., Sahabuddin, Z. A., & Trismadi, T. (2025). A comprehensive

meta-analysis of determinants influencing military expenditure: New methodological insights and implications for defence budget allocation. *Statistika, Politika i Polityka*, 16(1), 87–103. <https://doi.org/10.1515/spp-2024-0024>

Caldara, D., & Iacoviello, M. (2022). Measuring geopolitical risk. *American Economic Review*, 112(4), 1194–1225. <https://doi.org/10.1257/aer.20191823>

Cepparulo, A., & Pasimeni, P. (2024). *Defence spending in the European Union* (European Economy Discussion Paper No. 199). European Commission, Directorate-General for Economic and Financial Affairs. <https://doi.org/10.2765/883426>

George, J., & Sandler, T. (2022). NATO defense demand, free riding, and the Russo-Ukrainian war in 2022. *Journal of Industrial and Business Economics*, 49, 783–806. <https://doi.org/10.1007/s40812-022-00228-y>

Google. (n.d.). *Google Maps*. <https://maps.google.com>

Knezović, S., & Tkalec, M. (2025). Defense spending, conflict and economic growth in Europe. *Peace Economics, Peace Science and Public Policy*, 31(4), 461–486. <https://doi.org/10.1515/peps-2025-0004>

Lebacher, M., Thurner, P. W., & Kauermann, G. (2021). A dynamic separable network model with actor heterogeneity: An application to global weapons transfers. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 184(1), 201–226. <https://doi.org/10.1111/rssa.12620>

Nawrocki, K. (2023, September 18). *Guardians of foreign interests*. Institute of National Remembrance.

Pamp, O., Lebacher, M., Thurner, P. W., & Ziegler, E. (2021). Explaining destinations and volumes of international arms transfers: A novel network Heckman selection model. *European Journal of Political Economy*, 69, Article 102033. <https://doi.org/10.1016/j.ejpoleco.2021.102033>

Pedersen, R. B., Ohrt, A., & Svendsen, G. T. (2023). Free riding in NATO after the rise of Russia: Cost sharing, free riding and selective incentives in NATO from 2009 to 2019. *Journal of Transatlantic Studies*, 21, 54–72. <https://doi.org/10.1057/s42738-023-00109-9>

Stockholm International Peace Research Institute. (n.d.). SIPRI Arms Industry Database. <https://www.sipri.org/databases/armsindustry>

Stockholm International Peace Research Institute. (n.d.). SIPRI Arms Transfers Database. <https://www.sipri.org/databases/armstransfers>

Stockholm International Peace Research Institute. (n.d.). SIPRI Military Expenditure Database. <https://www.sipri.org/databases/milex>

Tian, N., Lopes da Silva, D., Béraud-Sudreau, L., Liang, X., Scarazzato, L., & Assis, A. (2023). Developments in military expenditure and the effects of the war in Ukraine. *Defence and Peace Economics*, 34(5), 547-562. <https://doi.org/10.1080/10242694.2023.2221877>

UNIAN. (2019, March 19). *Ukroboronprom refutes allegations of arms supplies to Russia*. <https://www.unian.info/economics/10484022-ukroboronprom-refutes-allegations-of-arms-supplies-to-russia.html>